UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE TECNOLOGIA
PROGRAMA DE PÓS-GRADUAÇÃO EM ENGENHARIA ELÉTRICA E DE COMPUTAÇÃO

Proposta de implementação paralela de algoritmo genético em FPGA

Matheus Fernandes Torquato

Orientador: Prof. Dr. Marcelo Augusto Costa Fernandes

Dissertação de Mestrado apresentada ao Programa de Pós-Graduação em Engenharia Elétrica e de Computação da UFRN (Área de concentração: Engenharia de Computação) como parte dos requisitos para obtenção do título de Mestre em Ciências.

Número de Ordem do PPgEEC: M507
Natal, RN, Dezembro de 2017

Universidade Federal do Rio Grande do Norte - UFRN

Divisão de Serviços Técnicos

Catalogação da publicação na fonte. UFRN / Biblioteca Central Zila Mamede

Torquato, Matheus Fernandes.
Proposta de implementação paralela de algoritmo genético em FPGA / Matheus Fernandes Torquato. - 2017.
74 f.: il.

Dissertação (mestrado) - Universidade Federal do Rio Grande do Norte. Centro de Tecnologia. Programa de Pós-Graduação em Engenharia Elétrica e de Computação.

Orientador: Marcelo Augusto Costa Fernandes

1. FPGA - Dissertação. 2. Hardware reconfigurável - Dissertação. 3. Algoritmo genético - Dissertação. 4. Processamento paralelo - Dissertação. 5. Inteligência artificial - Dissertação. I. Fernandes, Marcelo Augusto Costa. II. Título.

RN/UF/BCZM CDU 004

Proposta de implementação paralela de algoritmo genético em FPGA

Matheus Fernandes Torquato

Dissertação de Mestrado apresentada em 01 de Dezembro de 2017. Banca examinadora composta pelos seguintes membros:

Prof. Dr. Marcelo Augusto Costa Fernandes (Orientador) ..... DCA/UFRN

Prof. Dr. Ricardo Pezzuol Jacobi (Avaliador Externo) ..... UnB

Prof. Dr. José Alberto Nicolau de Oliveira ..... DEE/UFFN

Prof. Dr. Luiz Felipe De Queiroz Silveira ..... DCA/UFRN

Aos meus pais pelo suporte incondicional aos meus estudos não somente durante a realização desse trabalho, mas durante toda a minha vida.

À minha avó Iza (In memoriam)

Agradecimentos

Em primeiro lugar e acima de tudo, aos meus pais Boanerges e Rinalda, meu eterno agradecimento; Não cabem em palavras os agradecimentos aos meus espelhos de honestidade, caráter e numerosos outros adjetivos positivos;

Aos meus irmãos Diogo e Mariana, por serem um espelho de sucesso profissional;

Ao meu orientadaor, Professor Dr. Marcelo Augusto Costa Fernandes a quem tenho profunda admiração como profissional e pessoa, agradeço pela orientação, paciência, incentivo e ensinamentos de engenharia e de vida;

Aos colegas e amigos, agradeço pela companhia nessa jornada. Não citarei nomes para não correr o risco de pecar esquecendo alguém, porém nada se faz sem amigos ao redor;

À minha companheira Alice, pelo amor, carinho, afeto e suporte emocional. Obrigado por tornar os meus dias melhores. O árduo caminho ao seu lado parece mais fácil;

À instituição UFRN, pelo corpo docente de qualidade e pelo padrão de ensino;

À CAPES, pelo apoio financeiro.

Resumo

Os Algoritmos Genéticos (AGs) são utilizados para resolver problemas de busca e otimização no qual, uma solução ótima pode ser encontrada utilizando um processo iterativo e transições probabilísticas. Todavia, dependendo do tipo de problema, o tempo para encontrar a solução pode ser elevado em máquinas sequenciais devido à complexidade computacional do algoritmo genético. Assim, esse trabalho possui como objetivo o desenvolvimento de um protótipo associado a uma implementação paralela de um algoritmo genético em FPGA (*Field-programmable gate array*). O principal objetivo do desenvolvimento dessa arquitetura é a otimização do tempo de processamento do sistema. Resultados associados com o tempo de processamento e a área ocupada para vários tamanhos de população foram analisados. Estudos relativos à precisão da resposta do algoritmo genético para o problema de otimização de funções com uma e duas variáveis também foram analisados para a implementação em hardware. Todo projeto foi desenvolvido utilizando a plataforma de desenvolvimento *System Generator* da Xilinx tendo como FPGA alvo um Virtex-7 xc7vx550t-1ffg1158 FPGA.

Palavras-chave: FPGA, Hardware, Algoritmo Genético, Processamento Paralelo.

Abstract

Genetic Algorithms (GAs) are used to solve search and optimization problems in which an optimal solution can be found using an iterative process and using probabilistic transitions. However, depending on the type of problem, the time required to find a solution can be high in sequential machines due to the computational complexity of genetic algorithm. This work proposes a parallel implementation of a genetic algorithm on field-programmable gate array (FPGA). Optimization of the system's processing time is the main goal of this project. Results associated with the processing time and area occupancy (in FPGA) for various population size are analyzed. Studies concerning the accuracy of the GA response for the optimization of functions with one and two variables were also analyzed for the hardware implementation. The project was developed using the System Generator software (Xilinx development platform) and the Virtex-7 xc7vx550t-1ffg1158 FPGA.

Keywords: FPGA, Hardware, Genetic Algorithm, Parallel Processing.

Sumário

Sumário i

Lista de Figuras iii

Lista de Tabelas v

Lista de Símbolos, Variáveis e Siglas vii

1 Introdução 1
- 1.1 Trabalhos Relacionados . . . 3
- 1.2 Principais Contribuições . . . 5
- 1.3 Artigos Publicados . . . 6
- 1.4 Organização da Dissertação . . . 7

2 Computação Reconfigurável 9
- 2.1 Descrição do Capítulo . . . 9
- 2.2 Computação Reconfigurável . . . 9

3 Algoritmos Genéticos 17
- 3.1 Descrição do Capítulo . . . 17
- 3.2 História . . . 17
- 3.3 Visão Geral . . . 18
- 3.4 Função de Avaliação . . . 21
- 3.5 Seleção . . . 21
- 3.6 Cruzamento . . . 22
- 3.7 Mutação . . . 23

4 Descrição do Projeto 25
- 4.1 Descrição do Capítulo . . . 25
- 4.2 Descrição Geral do Projeto . . . 25
- 4.3 Módulo de Função de Avaliação - MFA . . . 26
- 4.4 Módulo de Seleção - MS . . . 28
- 4.5 Módulo de Cruzamento - MC . . . 29
- 4.6 Módulo de Mutação - MM . . . 31
- 4.7 Módulo de Sincronização - MSinc . . . 32

5 Resultados **35**
5.1 Descrição do Capítulo 35
5.2 Metodologia 35
5.3 Experimentos 36
5.4 Resultados 39
5.5 Comparações com trabalhos do estado da arte 43

6 Considerações Finais **47**
6.1 Conclusões 47
6.2 Perspectivas Futuras 48

Referências bibliográficas **49**

Lista de Figuras

2.1 Classificação de dispositivos de *hardware* quanto a suas velocidades e flexibilidades. Fonte: (Azarian & Ahmadi 2009). 10
2.2 Arquitetura básica de um FPGA. Fonte: (Floyd 2009). 11
2.3 Blocos lógicos configuráveis básicos (CLBs) dentro de interconexões programáveis globais em linha/coluna. Fonte: (Floyd 2009). 12
2.4 Organização básica de um módulo lógico em um CLB. Fonte: (Floyd 2009). 12
2.5 CLB simplificado em um FPGA Virtex. Fonte: (Floyd 2009). 14
2.6 LUT de 4 entradas implementando a função AND e a tabela verdade. Fonte: Autoria Própria 15
2.7 Fluxo do processo de síntese em um FPGA. Fonte: (da Silva 2016). . . . 16

3.1 Ilustração com os passos de um AG. Fonte: (Russell et al. 2003). 19
3.2 Exemplo de cruzamento de um AG onde os indivíduos são configurações das posições das peças em um jogo de xadrez. Fonte: (Russell et al. 2003). 23
3.3 Exemplo de uma mutação em um AG. Fonte: Autoria própria. 24

4.1 Arquitetura geral da proposta de implementação paralela de algoritmo. Fonte: Autoria própria. 26
4.2 Arquitetura do j-ésimo módulo de Função de Avaliação (MFAj). Fonte: Autoria própria. 27
4.3 Arquitetura do j-ésimo módulo de seleção (MSj). Fonte: Autoria própria. 28
4.4 Arquitetura do j-ésimo módulo de cruzamento (MC$_j$). Fonte: Autoria própria. 29
4.5 Arquitetura do j-ésimo módulo de cruzamento (MCPQ1$_j$). Fonte: Autoria própria. 30
4.6 Arquitetura do j-ésimo módulo de mutação (MM$_j$). Fonte: Autoria própria. 32
4.7 Arquitetura do Módulo de Sincronização - MSinc. Fonte: Autoria própria. 32

5.1 Função de Avaliação F1: $f(x) = x^3 - 15x^2 + 50$. Fonte: Autoria própria. . 36
5.2 Função de Avaliação F2: $f(x,y) = 8x - 4y + 1020$. Fonte: Autoria própria. 37
5.3 Função de Avaliação F3: $f(x,y) = \sqrt{x^2 + y^2}$. Fonte: Autoria própria. . . 38
5.4 Minimização da F1. Fonte: Autoria própria 39
5.5 Minimização da F3. Fonte: Autoria própria. 40
5.6 Ocupação dos Registradores no FPGA variando com N. Fonte: Autoria própria. 41
5.7 Ocupação das LUTs no FPGA variando com N. Fonte: Autoria própria. . 42
5.8 Queda de Rg ao variar m. Fonte: Autoria própria. 43

5.9 Relação entre o uso de LUTs com o aumente de m. Fonte: Autoria própria. 43

Lista de Tabelas

5.1 Resultados da síntese do algoritmo genético em FPGA para $m = 20$. . . . 40
5.2 Tabela comparativa com trabalhos do estado da arte 45

Lista de Símbolos, Variáveis e Siglas

Símbolos

$\approx$	Aproximadamente
$\wedge$	Operação binária AND
$\vee$	Operação binária OU
μs	Microssegundo
$\oplus$	Operação Ou Exclusivo
ms	Milissegundo
ns	Nanossegundo

Variáveis

δ	$\alpha(px)+\beta(qx)$
a	Número de *bits* de y
c	Número de *bits* de $\alpha(px)$ e $\beta(qx)$
d	Número de *bits* de δ
h	Cabeça do cromossomo na etapa de cruzamento
j	Índice da variável ou módulo
k	Número de iterações do AG
m	Número de *bits* de x, w e z
N	Tamanho da população do AG
P	Número de cromossomos a sofrerem mutação
pw	Primeira metade de w
px	Primeira metade de x
pz	Primeira metade de z

qw	Segunda metade de w
qx	Segunda metade de x
qz	Segunda metade de z
R_g	Taxa de amostragem da arquitetura apresentada
$R_{g\eta}$	Taxa de amostragem de uma arquitetura com η atrasos
t	Cauda do cromossomo na etapa de cruzamento
T_g	Tempo de cada geração k da arquitetura apresentada
$T_{g\eta}$	Tempo de cada geração k de uma arquitetura com η atrasos
W	Conjunto de w
w	Cromossomo selecionado
X	População do AG
x	Cromossomo do AG
Y	Conjunto de y
y	Valor de *fitness* do cromossomo
Z	Conjunto de z
z	Cromossomo pós-cruzamento

Siglas

AG	Algorítmo Genético
AM	Algorítmo Memético
ASIC	Circuito Integrado de Aplicação Específica, do inglês *Application Specific Integrated Circuits*
CLB	Bloco Lógico Configurável, do inglês *Configurable Logic Block*
CPLD	Dispositivo Lógico Programável Complexo, do inglês *Complex Programmable Logic Device*
CPU	Unidade Central de Processamento, do inglês *Central Processing Unit*
CR	Computação Reconfigurável
FA	Função de Avaliação

FC	Função de Cruzamento
FM	Função de Mutação
FPGA	Arranjo de Portas Programáveis em Campo, do inglês *Field Programmable Gate Array*
FS	Função de Seleção
GPP	Processadore de Uso Geral, do inglês *General Purpose Processor*
GPU	Unidade de Processamento Gráfico, do inglês *Graphics Processing Unit*
HDL	Linguagem de Descrição de *Hardware*, do inglês *Hardware Description Language*
IA	Inteligência Artifical
IP Core	Núcleo de Propriedade Intelectual
LFSR	Registrador de Deslocamento com Realimentação Linear, do inglês *Linear Feedback Shift Register*
LUT	*Look up Table*
MA	Módulo de Avaliação
MC	Módulo de Cruzamento
MHz	*Megahertz*
MM	Módulo de Mutação
MS	Módulo de Seleção
MSinc	Módulo de Sincronização
Msps	Milhões de amostras por segundo, do inglês *Mega samples per second*
RAM	Memória de Acesso Aleatório, do inglês *Random Access Memory*
ROM	Memória de Apenas Leitura, do inglês *Read-Only Memory*
TM	Taxa de Mutação

Traduções

Throughput	Quantidade de dados processados em um determinado espaço de tempo

Capítulo 1

Introdução

Nos últimos anos, o crescente aumento do número de aplicações críticas envolvendo sistemas de tempo real aliado ao aumento da densidade dos circuitos integrados e à redução progressiva da tensão de alimentação, tornou a tarefa de criar soluções computacionais adequadas cada vez mais árdua.

Devido à pressão do mercado de eletrônicos de consumo que exige altas velocidades de processamento em intervalos de tempo cada vez menores, sem deixar de lado a economia de energia, a indústria da tecnologia tem se deparado com um cenário extremamente competitivo e desafiador no que se refere ao projeto de soluções em *hardware* com o intuito de suprir essa crescente demanda.

Uma forma encontrada por pesquisadores e desenvolvedores para atacar tais demandas é utilizando técnicas de paralelização de algoritmos. O processamento paralelo é utilizado para processar dados concorrentemente fazendo com que enquanto a computação de um estágio do algoritmo é feita, outros estágios realizem operações igualmente válidas em um outro conjunto de dados (Rodriguez & Moreno 2015). Ao aliar a implementação em *hardware* à paralelização de algoritmos tem-se, na maioria das vezes, uma solução satisfatória para aplicações de alto desempenho e com maior velocidade quando comparadas às soluções seriais.

Os *Field Programmable Gate Arrays* (FPGAs), no português Arranjo de Portas Programável em Campo, são dispositivos de *hardware* reconfigurável adequados à esse cenário devido a natureza de sua arquitetura. Tendo em vista que os FPGAs são enormes arranjos de portas configuráveis, essas portas podem ser programadas para funcionar como diversos caminhos paralelos em *hardware*. Desta forma, tem-se uma paralelização real em que as operações realizadas não necessitam competir pelos mesmos recursos já que cada uma estará sendo executada por diferentes portas (Instruments 2011). A crescente densidade e redução do preço dos FPGAs aumentam as oportunidades para desenvolvedores e pesquisadores usarem dispositivos FPGA de maior densidade para implementações em *hardware* (Jewajinda & Chongstitvatana 2009) tendo em vista que o uso de tais dispositivos é vantajoso uma vez que o tempo de desenvolvimento e custo são reduzidos significativamente (Tirumalai et al. 2007).

Uma vez que tem-se essas ferramentas de *hardware* em mãos, uma boa aplicação para tirar proveito desse potencial de processamento é a utilização delas em algoritmos de busca e otimização e meta-heurísticas em geral. Meta-heurísticas são estratégias comu-

mente utilizadas para resolver problemas NP-Difíceis por oferecerem melhores soluções e geralmente com tempo de processamento menor do que por outros tipos de técnicas mais tradicionais.

A Classe de Problemas NP (tempo polinomial não-determinístico) é formada pelo conjunto de problemas que podem ser resolvidos em tempo polinomial em um computador não-determinístico. Um problema de decisão $A \in$ NP é NP-Completo se todos os outros problemas de NP se transformam polinomialmente em A. Assim, se existir um algoritmo (determinístico) polinomial para a resolução de algum problema NP-Completo, então todos os problemas da classe NP também poderão ser resolvidos em tempo polinomial. Assim, seria possível mostrar que P = NP. Há fortes evidências de que isso não é possível.

Um problema A pertence a classe NP-Difícil se existe um problema $B \in$ NP-Completo que pode ser transformado em A em tempo polinomial. Portanto, a única diferença com relação à classe NP-Completo é que A não é provado ser NP.

A otimização é o mecanismo utilizado para encontrar a melhor solução, ou solução ótima, de um conjunto de soluções para um problema. São problemas complexos, muitas vezes de difícil solução, e que envolvem significativas reduções de gastos, melhorias de tempo de processamento, ou uma melhor alocação de recursos em atividades. Em geral, problemas de busca e otimização tentam encontrar uma solução que, por exemplo, maximizam o lucro e minimizam o custo (Munakata 1998). Alguns exemplo de aplicações que utilizam esses algoritmos são: desenvolvimento e roteamento de circuitos integrados, planejamento de layout de fábricas, programação automatizada e otimização de redes de telecomunicações (Russell et al. 2003).

No presente trabalho a meta-heurística abordada será o algoritmo genético. O algoritmo genético é um poderoso método de busca e otimização estocástico que busca uma boa solução a partir de um grande espaço de busca (Lotfi et al. 2016). A versatilidade dos algoritmos genéticos pode ser demonstrada pela variedade de aplicações que incluem: planejamento de rotas em veículos aéreos não tripulados (VANTs) (Li & Duan 2012), codificação de sequências de DNA (Peng et al. 2016), diagnóstico de câncer de mama (Alzubaidi et al. 2016) e compensação de movimentos de translação na captura de vídeos (Karmali & Shelhamer 2006).

A junção de algoritmos genéticos com técnicas de paralelização e implementação em *hardware* reconfigurável resulta nesse trabalho que apresenta uma proposta de implementação paralela de um algoritmo genético em FPGA. Esse trabalho foca em aplicações críticas e de alto desempenho que necessitam de respostas com velocidades na ordem de microssegundos ou nanossegundos. Analisando de outra forma, em aplicações onde velocidade de processamento não seja o fator crítico ou seja menos limitante que a necessidade de baixo consumo de potência, é possível reduzir o consumo de energia do sistema, através da redução dos ciclos de *clock* sabendo-se que ao utilizar um *clock* menor do que a frequência de operação teórica máxima, a potência dinâmica diminui (Uyemura 2002).

Aplicações que processam um grande fluxo de dados podem ser beneficiadas e aceleradas por essa implementação aqui desenvolvida. Alguns exemplos dessas possíveis aplicações são: mineração de dados, internet táctil, *big data* e bioinformática.

1.1 Trabalhos Relacionados

Há tempos que os algoritmos genéticos e a Inteligência Artificial (IA) vêm sendo utilizada em aplicações das mais diversas para otimizar e encontrar soluções satisfatórias em problemas computacionais, de engenharia e de outros escopos. Mais recentemente, começou-se a observar uma gama maior de aplicações e variações dos algoritmos genéticos tais como aplicações paralelas e distribuídas, implementações em *hardware*, novas propostas para os operadores genéticos e implementações híbridas (*software* e *hardware*) de algoritmos genéticos.

Com o advento dos FPGAs em meados da década de 90 e sua popularização para aplicações de uso geral pós 2000 (Ionescu et al. 2015), observou-se um crescimento do número de aplicações de técnicas de IA em *hardware*, tendo em vista que com o advento dos dispositivos de *hardware* reconfigurável, os tempos de projeto e os custos foram reduzidos significativamente (Tirumalai et al. 2007).

No ano de (1995) apareceu a primeira implementação de algoritmo genético em FPGA, o HGA (Algoritmo genético baseado em *hardware*) proposto por (Scott et al. 1995). Só depois de algum tempo outras implementações semelhantes começaram a ser amplamente conhecidas. (Fernando et al. 2008) propuseram no ano de (2008) uma implementação de um núcleo de propriedade intelectual (*Intellectual Property Core - IP Core*) customizável para FPGA que implementou um algoritmo genético de propósitos gerais. O foco dos autores desse trabalho foi a programabilidade do algoritmo genético implementado no IP core. A customização pôde ser feita em termos do tamanho da população, número de gerações, taxa de cruzamento e mutação, semente do gerador de números aleatórios e da função de avaliação. Um dos destaques do trabalho foi o suporte à múltiplas funções de avaliação. O núcleo proposto suportou até oito diferentes funções de avaliação. Algumas destas funções de avaliação podem ser sintetizadas juntamente com o AG e implementadas no mesmo dispositivo FPGA. O núcleo proposto também possuía portas de entrada/saída adicionais que permitiam ao usuário adicionar mais funções de avaliação que foram implementadas em um segundo dispositivo FPGA ou em algum outro dispositivo externo. A implementação utilizou 13% das células lógicas de um Xilinx Virtex II Pro (xc2vp30-7ff896). No entanto, é clara a existência de um compromisso entre desempenho e flexibilidade e uma vez que os autores focaram na flexibilidade, a aceleração (*speedup*) em relação à implementação análoga em *software* foi de apenas $\times 5.16$.

Implementações de Algoritmos Genéticos em *hardware* também puderam ser vistas em (Oliveira & Júnior 2008), (Mengxu & Bin 2015), (Vavouras et al. 2009) e (Zhu et al. 2007). (Zhu et al. 2007) apresentou o OIMGA que em português quer dizer Algoritmo monogenético do indivíduo ótimo e é uma estratégia que retém apenas o indivíduo ideal fazendo com que os requisitos de memória sejam reduzidos drasticamente e exclui a necessidade de existência de um circuito para o cruzamento. (Oliveira & Júnior 2008) apresentaram uma implementação compacta de um algoritmo genético em FPGA que representou a população de cromossomos como um vetor de probabilidades. O trabalho focou no menor consumo de recursos de memória, energia e espaço em *hardware*, porém ele não foi implementado todo em FPGA e utilizou um *software* escrito em C++ para a computação dos valores na função de avaliação. (Vavouras et al. 2009) propuseram uma

implementação de um AG de alta velocidade em FPGA. O trabalho foi baseado no HGA proposto por (Scott et al. 1995), a primeira implementação de AG em FPGA conhecida, e após a etapa de experimentos os autores afirmam que o sistema desenvolvido superou qualquer solução existente ou proposta. O P-HGAv1, versão de (Vavouras et al. 2009) parametrizável e melhorada do HGA afirmou ter baixa necessidade de silício, suportar múltiplas funções de avaliação e ser parametrizável. Embora os autores tenham focado na velocidade do algoritmo e alcançado um tempo de 0.021 milissegundos para cada geração do AG, essa velocidade pode não ser compatível com aplicações em tempo real que exijam baixa latência.

(Santos & Alves 2010) propôs um algoritmo memético (AM) em FPGA aplicado ao problema do caixeiro viajante. O AM refina uma solução obtida pelo AG usando um procedimento de busca local, o que significa uma intensificação da busca que não acontece em um AG. Os autores afirmaram ter implementado o trabalho proposto e conseguido uma frequência de 96 MHz utilizando menos de 1% dos blocos lógicos do FPGA Xilinx Virtex-4 (XC4VLX80-10), com uma taxa de resposta (*throughput*) de uma solução a cada 13 nanossegundos, aproximadamente. No entanto, apesar de ser uma implementação em FPGA, o artigo não apresentou detalhes sobre a implementação em *hardware* nem uma análise dos recursos utilizados no FPGA.

Os trabalhos apresentados por (Mingas et al. 2012), (Ionescu et al. 2015) e (Qu et al. 2013) apresentaram aplicações para robôs móveis terrestres que utilizam AGs, sendo esses dois primeiros implementações embarcadas em FPGA. (Mingas et al. 2012) desenvolveram, de acordo com os autores, a primeira implementação em *hardware* de localização e mapeamento simultâneos para robôs terrestres baseado em AG. Os autores conseguiram uma aceleração de *hardware* significativa em comparação com a implementação de *software*, explorando os recursos de *pipelining* e paralelização do *hardware* reconfigurável. Nesse projeto os genes que constituíram a população genética representam movimentos possíveis do robô de sua posição anterior. Já no trabalho desenvolvido por (Ionescu et al. 2015), a aplicação teve como intuito determinar os movimentos ótimos considerando vários aspectos como rastreamento de rotas e baixo consumo de energia, evitando a colisão com obstáculos. Os autores frisaram que a implementação é adequada para utilização em tempo real e afirmam que todos os estágios do AG foram implementados em módulos de *hardware*. A solução apresentada neste trabalho ofereceu um tempo de convergência inferior a 2 milissegundos, utilizou 17124 das 17600 *Lookup tables* (LUTs) presentes no FPGA, porém não foi informada a frequência obtida pela aplicação pós síntese no *hardware* reconfigurável. (Qu et al. 2013) desenvolveram um algoritmo genético com estratégia co-evolucionária para planejamento de trajetória global de vários robôs móveis. De acordo com (Koza 1991), co-evolução é o processo de adaptação mútua de duas ou mais populações, e é usado para refletir o fato de que todas as espécies estão simultaneamente co-evoluindo em um dado ambiente físico. A implementação de (Qu et al. 2013) afirmou uma melhora nos operadores genéticos de AGs convencionais e propôs um novo operador de modificação genética, porém esses desenvolvimentos não são feitos em *hardware*.

As implementações vistas em (Merabti & Massicotte 2014), (Sehatbakhsh et al. 2013) e (Chen & Wu 2011) são aplicações de AG em processamento digital de sinais e siste-

mas de controle feitos em FPGA. (Merabti & Massicotte 2014) apresentaram um AG em tempo real para aplicações de filtragem adaptativa. Além disso, foi apresentado em detalhes todos os módulos implementados em *hardware* de função de avaliação, seleção, cruzamento, mutação e gerador de números aleatórios utilizando registrador de deslocamento com realimentação linear (LFSR - *Linear Feedback Shift Register*). A implementação foi feita em *hardware* e após sua sintetização, alcançou uma taxa de 320 KGerações/segundo. Já (Sehatbakhsh et al. 2013) propuseram um AG para sistemas dinâmicos multi-portadoras baseados em bancos de filtros. Os autores de (Chen & Wu 2011) propuseram um projeto e implementação de um controlador PID baseado em AG e FPGA. Os autores afirmaram que o método de projeto do controlador PID inteligente baseado em FPGA e AG foi verificado com sucesso, e teve algumas vantagens, tais como design flexível, sintonização automática online, alta confiabilidade, curto ciclo de desenvolvimento técnico e velocidade de execução elevada. Para esse caso, cada indivíduo ou cromossomo da população do AG foi codificado com o conjuntos de ganhos Kp, Ki e Kd do controlador. Detalhes de área ocupada no FPGA e velocidades obtidas não foram informadas.

Por fim, (Guo et al. 2016) e (Lotfi et al. 2016) apresentaram formas paralelas e distribuídas de AG utilizando FPGAs. (Guo et al. 2016) propuseram resolver o problema da criação e execução de algoritmos genéticos paralelos em múltiplos FPGAs. O uso de múltiplas populações em AGs paralelos baseia-se na idéia de que o isolamento das populações pode manter uma maior diversidade genética, enquanto a comunicação entre elas pode fazer com que os AGs trabalhem em conjunto para encontrar boas soluções. A implementação de (Guo et al. 2016) foi aplicada a três diferentes *benchmarks*, incluindo o problema do caixeiro viajante, e os autores afirmaram, a partir dos resultados dos experimentos, que em uma configuração com 4 FPGAs alcançou uma média de aceleração de 30 vezes sobre um AG implementado em processador com múltiplos núcleos. (Lotfi et al. 2016) apresentaram o GRATER, uma dinâmica de projetos automatizada para aceleradores em FPGA que aproveitam a computação imprecisa para aumentar o paralelismo de dados e alcançar maior rendimento computacional. Nesse trabalho a ideia principal foi a seguinte: uma vez que a redução na área em *hardware* resulta em uma melhor utilização do paralelismo de dados e, desse modo, aumenta o rendimento, faz-se uma negociação através das técnicas de projeto de circuito envolvendo área/energia/desempenho em troca de qualidade de resultados através de implementações imprecisas de blocos específicos de *hardware* como somador e multiplicador. Ainda em (Lotfi et al. 2016), a programação genética foi utilizada para evoluir variantes do *kernel* de entrada até que seja encontrado um com atribuições ideais que reduzam a área do *kernel* sintetizada enquanto ainda satisfaçam estocasticamente a qualidade de resultado desejada. O resultado da síntese em um FPGA Altera Stratix V mostra que a área reduzida dos *kernels* aproximados produzem um rendimento de $1,4$ à $3,0$ vezes superior com menos de 1% de perda de qualidade em relação a execução de *benchmarks*.

1.2 Principais Contribuições

Esta dissertação de mestrado apresenta como contribuição uma proposta de arquitetura paralela de algoritmo genético em FPGA. Como afirmado por (Mengxu & Bin 2015),

existem duas formas de aumentar a velocidade de funcionamento de um algoritmo genético: utilizando uma configuração paralela do algoritmo e embarcando essa solução em *hardware*. A paralelização do fluxo de dados permite que a arquitetura aqui proposta seja utilizada em aplicações onde haja uma grande quantidade de dados a serem processadas e restrições rigorosas do tempo de processamento, onde seria inviável o uso de processadores genéricos, como CPUs ou até mesmo GPUs.

(Vavouras et al. 2009) desenvolveram uma implementação de um Algoritmo Genético em FPGA de alta velocidade, no entanto, como demonstraram os seus experimentos, o projeto só funcionou com funções de uma variável, o que limita o uso da aplicação a poucos problemas práticos. Além disso, a implementação utilizou um processador embarcado (um *PowerPC*), adicionando um atraso desnecessário ao design. Conforme apresentado no Capítulo 4, a implementação apresentada nessa dissertação é capaz de lidar com uma gama muito maior de funções usando exclusivamente componentes de *hardware*.

Por sua vez, a implementação apresentada por (Fernando et al. 2008) suporta múltiplas funções de *fitness*, bem como programação de parâmetros do AG. No entanto, não foram apresentados detalhes suficientes sobre a implementação do *hardware*, impedindo assim qualquer replicação pela comunidade científica. Conforme será apresentado no Capítulo 4, os detalhes de todos os módulos aqui implementados são mostrados permitindo sua replicação.

Observou-se que (Qu et al. 2013), (Guo et al. 2016) e (Lotfi et al. 2016) apresentaram a mesma desvantagem que em (Fernando et al. 2008) e que (Zhu et al. 2007) tiveram a mesma restrição de funções de apenas uma variável como em (Vavouras et al. 2009).

Essencialmente, este trabalho agrega todos os importantes recursos de avaliação e implementação encontrados nas obras relacionadas e suas contribuições principais podem ser resumidas no seguinte:

- Implementação paralela em *hardware* na sua totalidade sem processadores ou *softwares* embarcados adicionais resultando em ganhos de desempenho (*speedup*) em relação a outros trabalhos do estado da arte;
- Uma análise de frequência, velocidade e área ocupada no FPGA baseada nos dados fornecidos pelos relatórios gerados pela síntese no *hardware*. Diferente dos trabalhos encontrados, o presente trabalho apresenta uma avaliação quantitativa detalhada de acordo com a variação de parâmetros chave da implementação;
- Síntese em FPGA da arquitetura desenvolvida para diferentes tamanhos de população do AG em ponto fixo.

1.3 Artigos Publicados

1. Matheus F. Torquato e Marcelo A. C. Fernandes. ”Proposta de Implementação Paralela de Algoritmo Genético para Funções de Duas Variáveis em FPGA”. *Em: 9a edição da Escola Potiguar de Computação e suas Aplicações (EPOCA 2016 - Natal-RN).*
2. Lucileide M. D. da Silva, Matheus F. Torquato e Marcelo A. C. Fernandes. ”Proposta de Arquitetura em Hardware para FPGA da Técnica Q-learning de Apren-

dizagem por Reforço". *Em: XIII Encontro Nacional de Inteligência Artificial e Computacional (ENIAC 2016 - Recife-PE).*

3. Matheus F. Torquato e Marcelo A. C. Fernandes. "Proposta de Implementação Paralela de Algoritmo Genético em FPGA". *Em: XXI Congresso Brasileiro de Automática (CBA 2016 - Vitoria, ES).*

1.4 Organização da Dissertação

A presente dissertação está organizada da seguinte forma: Nesse primeiro Capítulo foi apresentada uma introdução, motivação e contextualização do trabalho. Além disso, foi feita uma detalhada revisão bibliográfica contendo trabalhos relacionados que contemplam o estado da arte da área aqui abordada. Por fim, foram explicitadas as principais contribuições fornecidas à comunidade científica por esse trabalho além dos artigos já publicados.

No Capítulo 2 será apresentada uma fundamentação teórica sobre computação reconfigurável onde serão apresentados detalhes da arquitetura do FPGA, *hardware* utilizado para a implementação da arquitetura proposta.

No Capítulo 3 será apresentada uma fundamentação teórica sobre a meta-heurística que será utilizada, os algoritmos genéticos, explorando as principais características, vantagens e diferentes aplicações.

O Capítulo 4 irá apresentar uma descrição detalhada do desenvolvimento e da implementação da arquitetura, descrevendo os diversos módulos em *hardware* utilizados para a construção do algoritmo genético paralelo.

Adiante, no Capítulo 5, será feita uma análise criteriosa dos resultados obtidos a partir da implementação descrita no Capítulo 4. Resultados de simulação, de síntese no FPGA e a validação da arquitetura proposta será apresentada. A análise será feita para parâmetros como a área de ocupação e a frequência de amostragem, levando em conta diferentes configurações da arquitetura proposta e embarcada no *hardware* reconfigurável.

Finalmente, no Capítulo 6 serão apresentadas considerações finais, conclusões sobre os resultados obtidos e perspectivas futuras de trabalho.

Capítulo 2

Computação Reconfigurável

2.1 Descrição do Capítulo

Esse Capítulo irá, inicialmente, fazer uma breve descrição sobre sistemas de computação reconfigurável (CR) juntamente com uma contextualização de como e onde esses dispositivos se encaixam no escopo de *hardwares* computacionais. Em seguida, os FPGAs serão destacados dentre os dispositivos de CR e suas principais características serão descritas, assim como detalhes da arquitetura genérica dos FPGAs serão apresentados através de Figuras e esquemáticos.

2.2 Computação Reconfigurável

O conceito de computação reconfigurável existe há muito tempo. Até mesmo os processadores de uso geral utilizam-se das mesmas ideias básicas tais como a reutilização de componentes computacionais para cálculos independentes e a utilização de multiplexadores para controlar o roteamento entre esses componentes. No entanto, o termo "Computação Reconfigurável"utilizado nos dias atuais se refere a sistemas que incorporam algum tipo de customização de *hardware* que altera o modo como o sistema é utilizado a partir de pontos físicos de controle. Estes pontos de controle podem ser alterados periodicamente, a fim de executar diferentes aplicações usando o mesmo *hardware* (Compton & Hauck 2002).

No escopo dos *hardwares* computacionais, a computação reconfigurável é uma das áreas mais importantes de pesquisa na atualidade. De acordo com (Compton & Hauck 2002), a pesquisa em computação reconfigurável está se tornando cada vez mais atrativa devido ao seu potencial de acelerar uma ampla variedade de aplicações. A sua característica principal é a capacidade de realizar computações em *hardware* para aumentar o desempenho na execução, enquanto retém parte da flexibilidade de uma solução em *software*. Segundo (Todman et al. 2005), projetos utilizando computação reconfigurável são capazes de alcançar um *speedup* de até 500 vezes e 70% de economia de energia em comparação a implementações de microprocessadores para aplicações específicas. Esse modelo de computação destina-se a preencher a lacuna entre *hardware* e *software*, alcançando um desempenho muito maior comparado ao de aplicações em *software* enquanto mantém um maior nível de flexibilidade do que soluções em *hardware*.

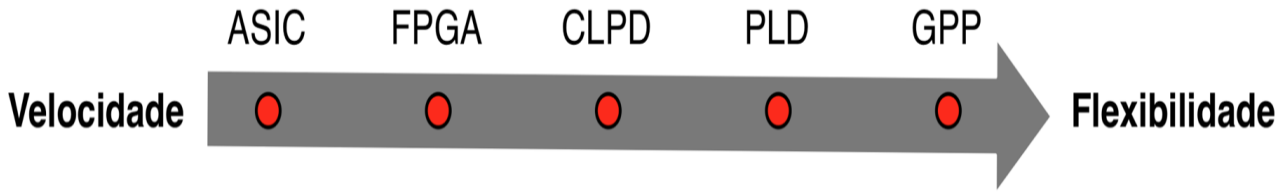


Figura 2.1: Classificação de dispositivos de *hardware* quanto a suas velocidades e flexibilidades. Fonte: (Azarian & Ahmadi 2009).

A computação reconfigurável utiliza-se de dispositivos reconfiguráveis como os FPGAs, por exemplo, para realizar as computações necessárias. Computação reconfigurável é também conhecida como computação configurável ou computação customizável já que muitas das técnicas de projeto podem ser vistas como uma personalização de um dispositivo computacional fabricado para aplicações específicas.

A Figura 2.1 (Azarian & Ahmadi 2009) mostra a classificação de algumas soluções computacionais em *hardware*, algumas não reconfiguráveis, de acordo com suas respectivas velocidades e flexibilidades. Pode-se notar que os circuitos integrados de aplicação especifica, do inglês ASIC, lideram como a solução mais veloz. Na outra extremidade dos dispositivos mais flexíveis, os processadores de propósito geral, do inglês GPP. Eles são bastante flexíveis pois podem executar uma imensa quantidade de funções e todas elas podem ser criadas por combinação das várias instruções suportadas pelo processador.

Nesse trabalho, o foco será voltado para os FPGAs: dispositivos de *hardware* reconfigurável que possuem flexibilidade suficiente para as tarefas exigidas nesse projeto e possuem um desempenho melhor do que todas as outras soluções de *hardware* reconfigurável. A principal diferença entre os circuitos do tipo ASIC e os circuitos reconfiguráveis, como os FPGAs, é justamente a possibilidade de adaptar o *hardware* através da reprogramação, sendo desnecessária a fabricação de um novo chip a cada mudança na arquitetura do circuito integrado.

A diferença básica entre FPGAs e CPLDs, acrônimo para Dispositivo Lógico Complexo Programável, reside no método de interligação das células e no tipo de blocos lógicos fundamentais. A arquitetura de uma CPLDs consiste em blocos lógicos do tipo PAL/GAL ou PLA com interconexões programáveis.

FPGAs tornaram-se um dos principais meios de implementação de circuito digital durante a última década por causa de uma série de vantagens que esses dispositivos trazem consigo. Uma dessas principais vantagens é a sua arquitetura, Figura 2.2 (Floyd 2009), que afeta positivamente o desempenho, eficiência em área e consumo de energia no projeto final.

Embora os FPGAs sejam ótimas soluções quando se procura velocidade e, acima de tudo, flexibilidade, quando comparados a um ASIC a sua natureza flexível vem a um custo significativo se tratando de área de ocupação, atraso e consumo de energia. Um FPGA requer aproximadamente 20 a 35 vezes mais área do que os circuitos integrados de aplicação especifica, tem um desempenho em velocidade de aproximadamente 3 a 4

vezes mais lento do que um ASIC e consome cerca de 10 vezes mais energia dinâmica.

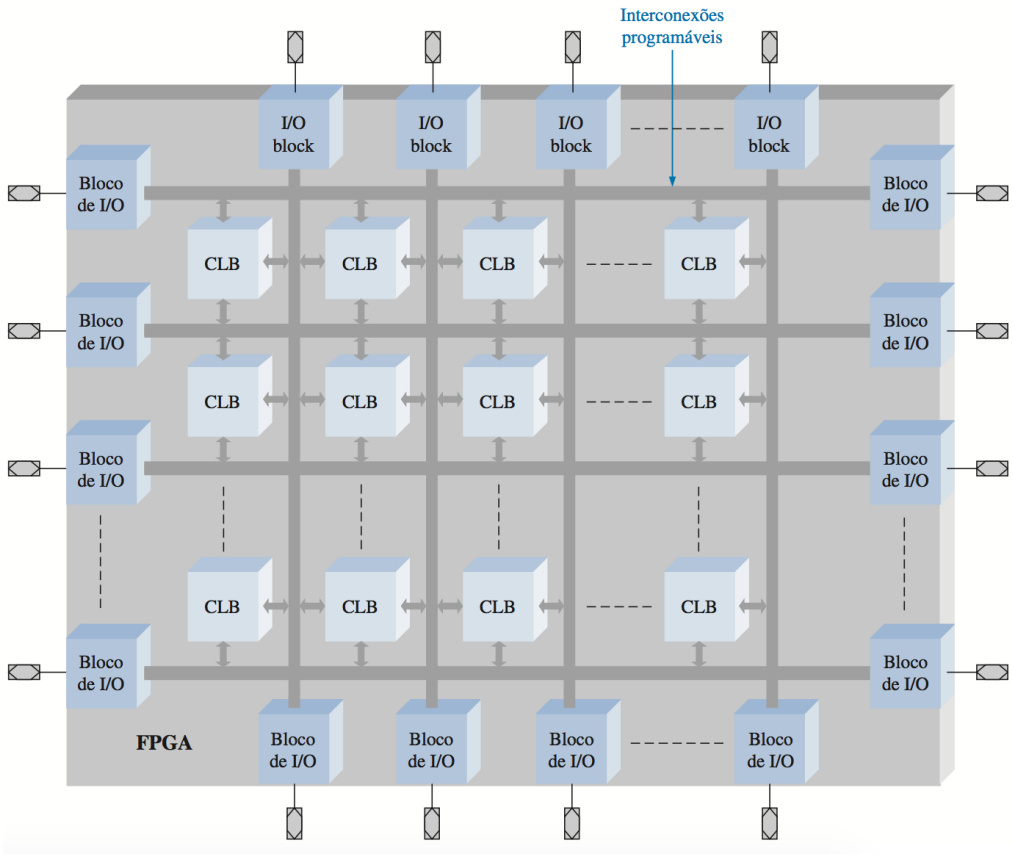


Figura 2.2: Arquitetura básica de um FPGA. Fonte: (Floyd 2009).

Os FPGAs são dispositivos de silício pré-fabricados que contém uma coleção de unidades de processamento configuráveis alocados em uma rede de interconexões configurável e que podem ser eletronicamente programadas para funcionar como praticamente qualquer tipo de circuito digital. Como mostrado na Figura 2.2, os três elementos básicos de um FPGA são os blocos lógicos configuráveis (CLB), as interconexões e os blocos de entrada/saída (I/O) (Floyd 2009).

Cada CLB é formado de múltiplos módulos lógicos menores e uma interconexão programável local que é usada para interconectar módulos dentro de uma CLB. Os blocos lógicos configuráveis podem ser usados como uma LUT, memória RAM ou registrador de deslocamento. A Figura 2.3 mostra a estrutura básica do CLB e a Figura 2.4 mostra um diagrama apresentando a estrutura de um módulo lógico contido dentro de um CLB. O módulo lógico exemplificado na Figura 2.4 é baseado em LUT. A tabela de busca LUT é um tipo de memória programável usada para gerar funções lógicas combinacionais do tipo soma-de-produtos.

Pesquisas na Universidade de Toronto (Rose et al. 1990) indicam que LUTs com quatro entradas produzem células lógicas com máxima eficiência em área. Além de uma LUT, um módulo lógico geralmente contém um *flip-flop* do tipo D (DFF), multiplexador

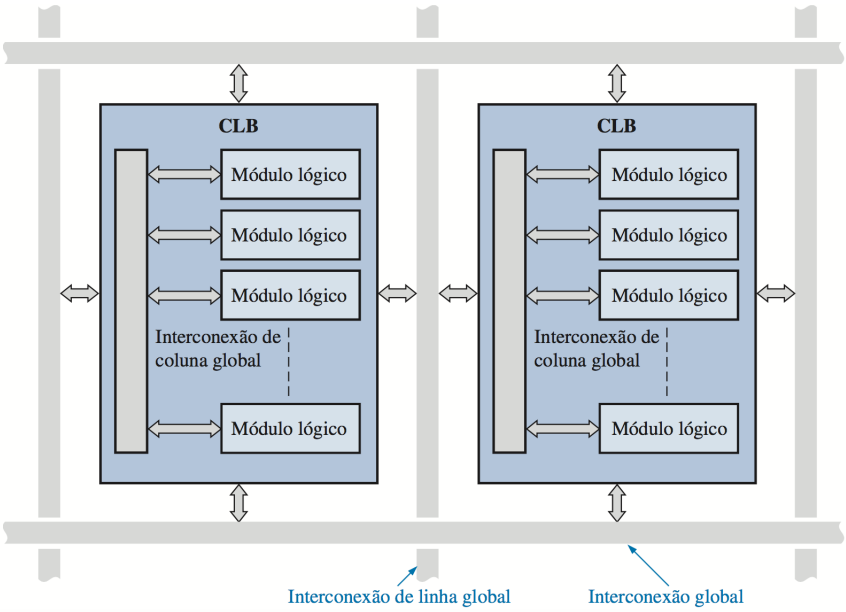


Figura 2.3: Blocos lógicos configuráveis básicos (CLBs) dentro de interconexões programáveis globais em linha/coluna. Fonte: (Floyd 2009).

e um bit de transporte para as operações aritméticas.

É importante notar que a arquitetura específica de cada FPGA varia de acordo com o fabricante. A área lógica configurável (denominada estrutura FPGA) da maioria das FPGAs Xilinx é dividida em blocos lógicos configuráveis (CLBs) sendo que cada CLB contém múltiplas unidades lógicas básicas denominadas de células lógicas (LCs). Cada célula lógica é baseada na lógica LUT de quatro entradas tradicional mais uma lógica adicional e um flip-flop. Uma LUT de 4 entradas pode produzir de um termo-produto a uma função de termos-produto com dezesseis termos. Duas células lógicas idênticas são denominadas de *slice* (fatia). A Figura 2.5 ilustra os níveis da lógica configurável da célula lógica do CLB. As densidades variam de cerca de 2000 a mais de 74.000 células

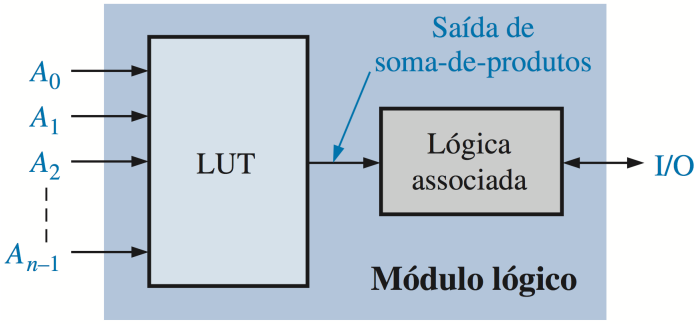


Figura 2.4: Organização básica de um módulo lógico em um CLB. Fonte: (Floyd 2009).

lógicas num único dispositivo Virtex (Floyd 2009).

A Figura 2.6 mostra uma LUT de 4 entradas implementando a função AND. Também é possível ver a tabela verdade do circuito digital de acordo com todas as possíveis entradas.

A criação de um circuito baseado em FPGA é um processo de criação de um fluxo de dados em formato binário (*bitstream*) que deve ser carregado no dispositivo reconfigurável, processo chamado síntese. Esse processo é geralmente especificado por uma linguagem de descrição de *hardware* (ou *Hardware Description Language* - HDL).

As ferramentas para realizar a síntese são desenvolvidas e fornecidas pelos fabricantes dos FPGAs. A partir do HDL é feita a síntese lógica que converte um código comportamental de alto nível em portas lógicas, em seguida, o mapeamento separa as portas em grupos que melhor se adequam aos recursos lógicos do FPGA, como pode ser visto na Figura 2.7 (da Silva 2016). O posicionamento (*placement*) atribui os agrupamentos lógicos para blocos lógicos específicos e o roteamento determina os recurso de interconexão que irão transportar os sinais. É importante salientar que esse posicionamento pode ser feito com foco na otimização de área ocupada ou da velocidade. Finalmente, com a geração do *bitstream*, um arquivo binário que define todos os pontos de programação do FPGA para configurar os blocos lógicos e de roteamento, o dispositivo reconfigurável poderá ser programado e estará pronto para executar os comandos contidos inicialmente no código fonte (Hauck & DeHon 2010).

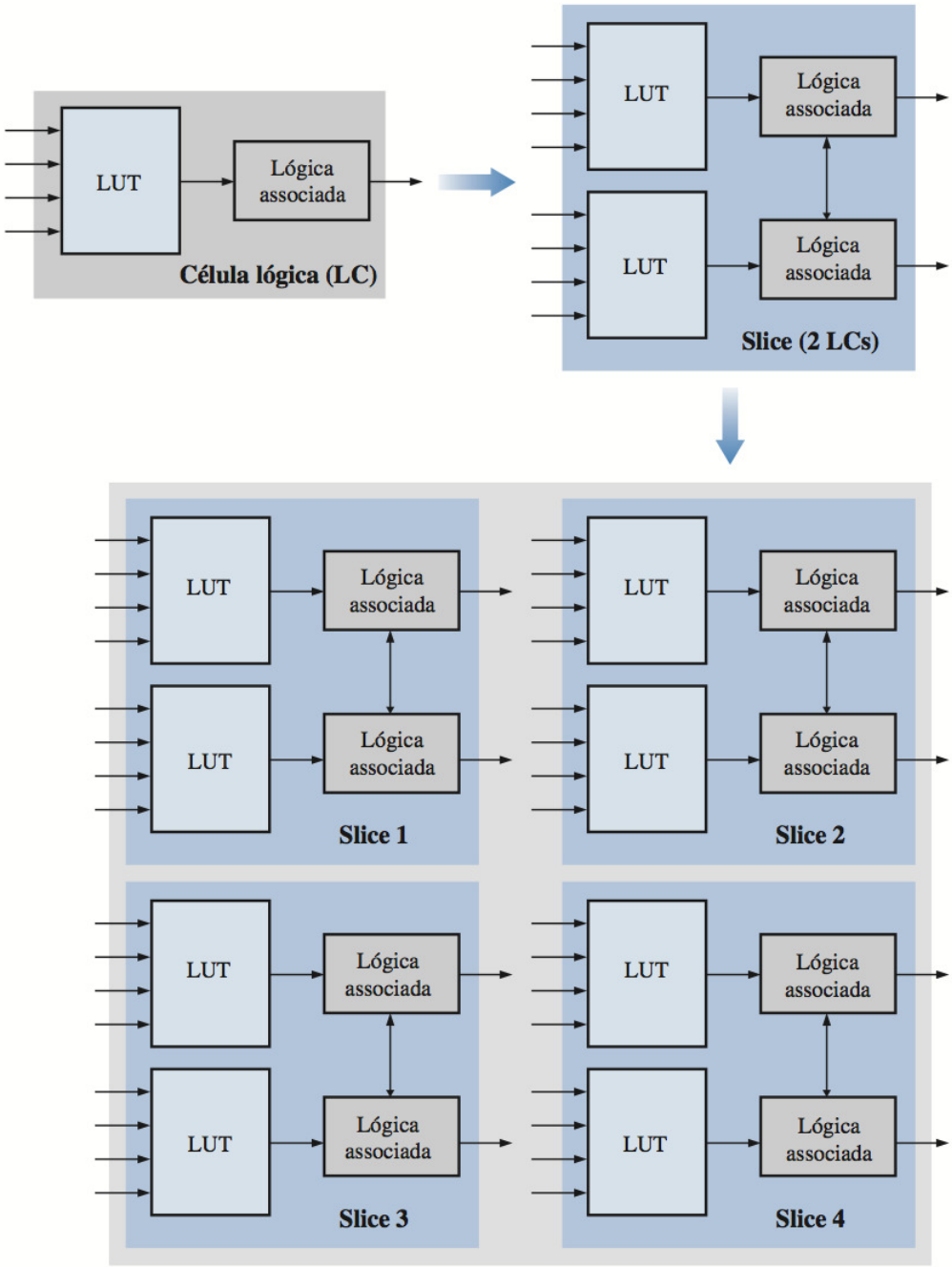
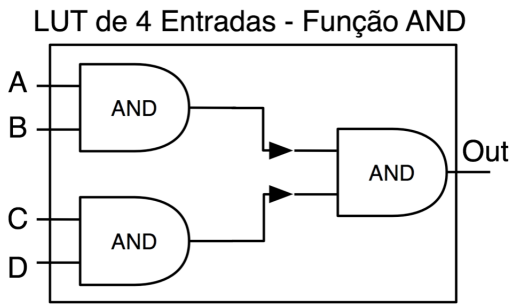


Figura 2.5: CLB simplificado em um FPGA Virtex. Fonte: (Floyd 2009).



a	b	c	d	(a∧b)∧(c∧d)
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

Figura 2.6: LUT de 4 entradas implementando a função AND e a tabela verdade. Fonte: Autoria Própria

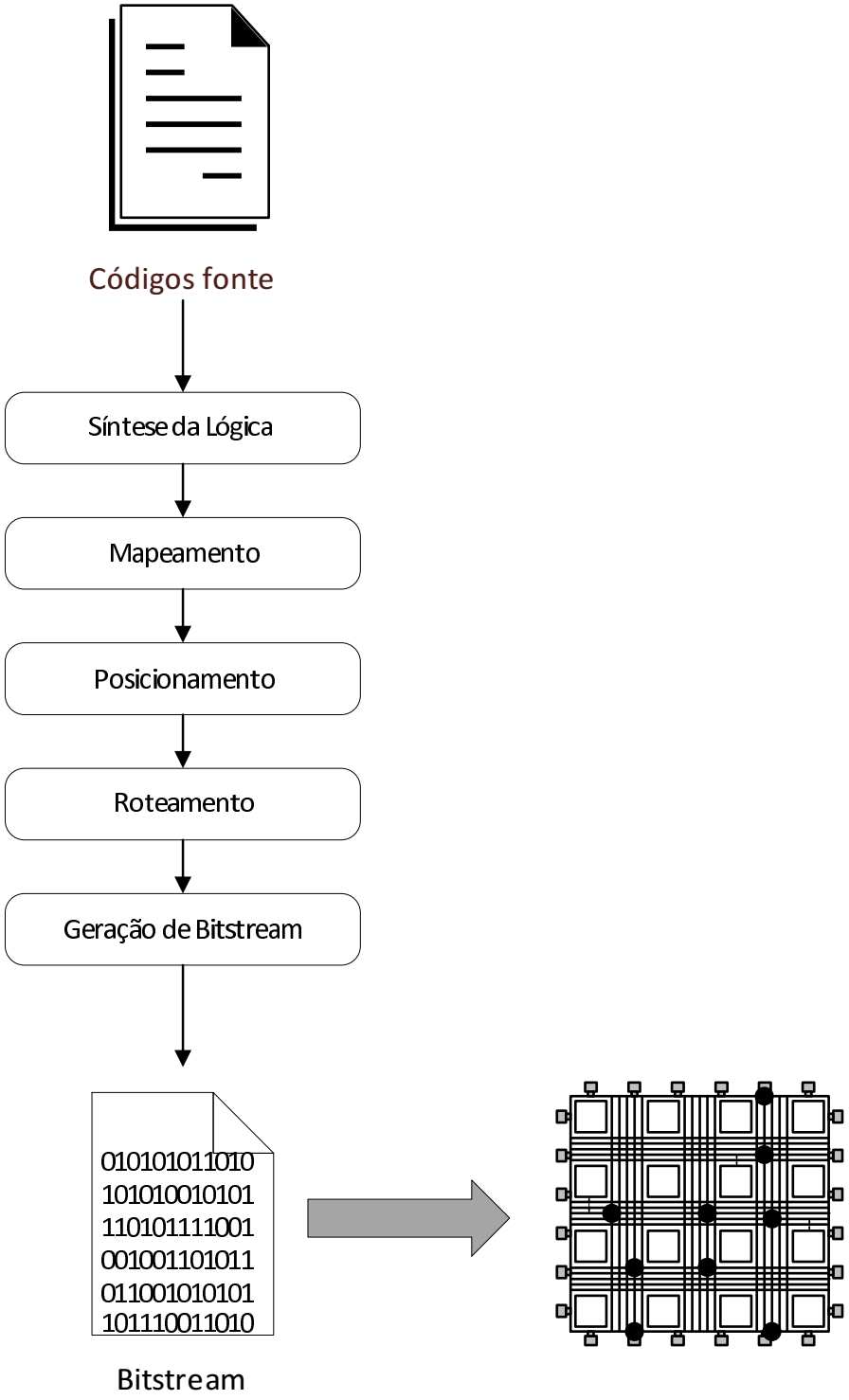


Figura 2.7: Fluxo do processo de síntese em um FPGA. Fonte: (da Silva 2016).

Capítulo 3

Algoritmos Genéticos

3.1 Descrição do Capítulo

Esse Capítulo abordará a meta-heurística dos Algoritmos Genéticos. Inicialmente, será feita uma contextualização histórica desses algoritmos. Em seguida, uma visualização geral da técnica será apresentada juntamente com ilustrações e um pseudocódigo. Por fim, as sessões seguintes serão responsáveis por descrever em detalhes as etapas de função de avaliação, seleção cruzamento e mutação do Algoritmo Genético.

3.2 História

Nas décadas de 1930 e 1940, os conhecimentos sobre a genética foram unidos às ideias evolucionistas de Charles Darwin numa síntese que teve como resultado uma teoria firme e embasada para explicar as leis que regem o processo evolutivo de seres vivos (Amabis & Martho 2010). Essa teoria ficou conhecida como teoria moderna da evolução, ou teoria sintética ou, ainda, Neodarwinismo.

Segundo o Neodarwinismo, os preceitos básicos do processo de evolução natural das espécies seriam:

1. Indivíduos da mesma ou diferentes espécies disputam por limitados recursos presentes no meio ambiente;
2. Dentre os vários concorrentes presentes em um meio, alguns, por conta de suas características específicas, possuem uma maior probabilidade de sobrevivência. Tais indivíduos são ditos mais adaptados ao ambiente;
3. Indivíduos mais adaptados possuem uma maior probabilidade de sobrevivência, e consequentemente reprodução;
4. Visto que no processo de reprodução um grande número de características dos pais são repassadas aos filhos, indivíduos que se reproduzem mais, tendem a propagar mais amplamente suas características nas gerações futuras;
5. Ao longo do processo de evolução, características mais desejáveis tendem a se propagar na espécie, aumentando assim o grau de adaptação como um todo;
6. O processo de reprodução não ocorre sem falha. Durante a replicação e transmissão dos genes aos novos indivíduos, criados o fenômeno conhecido como mutação pode

ocorrer. Este fenômeno é geralmente prejudicial ao indivíduo, mas em alguns casos pode incorporar a ele uma característica desejável não contida no conjunto de genes dos seus pais. Desta forma a natureza adquire a capacidade de explorar um número maior de combinações e possibilidades.

Os Algoritmos Genéticos são inspirados neste processo de evolução natural e são utilizados para resolver problemas de busca e otimização encontrados no mundo real utilizando um processo estocástico e iterativo.

É importante considerar que os AGs diferem dos algoritmos tradicionais de otimização nos seguintes aspectos:

- Baseiam-se em uma codificação do conjunto das soluções possíveis, e não nos parâmetros da otimização em si;
- Os resultados são apresentados como uma população de soluções e não como uma solução única;
- Não necessitam de nenhum conhecimento derivado do problema, apenas de uma forma de avaliação do resultado;
- Usam transições probabilísticas e não regras determinísticas.

3.3 Visão Geral

Os AGs são utilizados para resolver problemas de busca e otimização no qual, uma solução ótima pode ser encontrada utilizando um processo iterativo cuja busca se dá a partir de uma população inicial, que com a combinação dos melhores representantes, obtém-se uma nova população, que substitui a anterior (Holland 1975).

O AG é um algoritmo iterativo que é iniciado a partir de uma população de N indivíduos, criados aleatoriamente sendo N par, nesse trabalho, para facilitar a implementação. Em cada k-ésima iteração, também chamada de geração, os N indivíduos são avaliados, selecionados, recombinados e mutados para formar uma nova população também de N indivíduos, ou seja, toda a população de pais é substituída pela nova população de filhos. A nova população então é utilizada como entrada para a próxima iteração (geração) do algoritmo, e esse procedimento de atualização dos valores da geração anterior é repetido k vezes, onde k é número de gerações do algoritmo.

É importante explicitar que em um AG, um cromossomo da população pode assumir as mais diversas formas, como por exemplo, *string* de bits de números binários, números inteiros, *string* de caracteres, coordenadas em plano cartesiano, etc. A Figura 3.1 (Russell et al. 2003) apresenta uma ilustração resumida dos processos que ocorrem em cada geração do AG.

Pode-se observar na Figura: a) Indivíduos na forma de números inteiros em uma população de $N = 4$ indivíduos. b) Valor de cada indivíduo na função de avaliação e a respectiva probabilidade de cada indivíduo ser escolhido pelo método probabilístico da roleta. c) seleção dos indivíduos mais aptos e escolha aleatória do ponto de corte do número inteiro. Importante notar que um indivíduo foi selecionado duas vezes enquanto um outro não foi selecionado nenhuma vez. d) cruzamento entre os pais para geração dos novos indivíduos filhos. e) mutação.

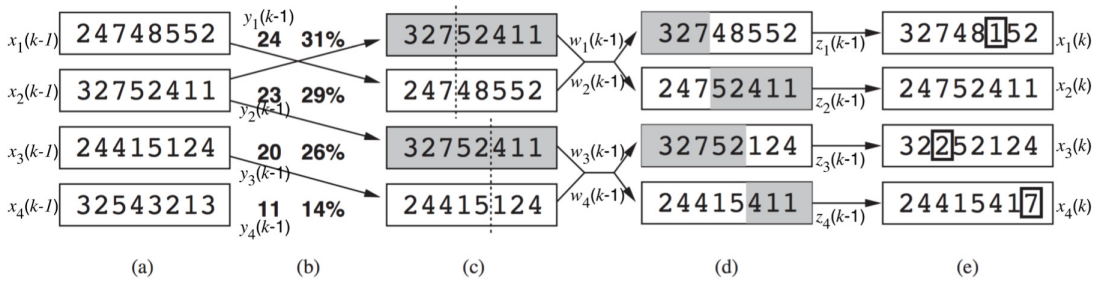


Figura 3.1: Ilustração com os passos de um AG. Fonte: (Russell et al. 2003).

Algorithm 1 Algoritmo Genético

1: Inicializa $\mathbf{X}[m](k)$ com indivíduos aleatórios
2: **for** $k \leftarrow 1$ **até** K **do**
3: **for** $j \leftarrow 1$ **até** N **do**
4: $y_j[a](k) \leftarrow FA\left(x_j[m](k)\right)$
5: **end for**
6: **for** $j \leftarrow 1$ **até** N **do**
7: $w_j[m](k) \leftarrow FS\left(\mathbf{Y}[a](k), \mathbf{X}[m](k)\right)$
8: **end for**
9: **for** $i \leftarrow 1$ **até** $N/2$ **do**
10: $\begin{bmatrix} z_{2i-1}[m](k) \\ z_{2i}[m](k) \end{bmatrix} \leftarrow FC\left(\begin{bmatrix} w_{2i-1}[m](k) \\ w_{2i}[m](k) \end{bmatrix}\right)$
11: **end for**
12: **for** $v \leftarrow 1$ **até** P **do**
13: $x_v[m](k) \leftarrow FM\left(z_v[m](k)\right)$
14: **end for**
15: **end for**

O Algoritmo 1 apresenta o pseudo-código do funcionamento do AG. Esse código detalha todas as variáveis e procedimentos que serão utilizados na implementação a ser apresentada nos capítulos seguintes. A variável $x_j[m](k)$ representa o j-ésimo individuo de m bits na k-ésima geração e $\mathbf{X}[m](k)$ é um vetor que armazena todos os N indivíduos, ou seja,

$$\mathbf{X}[m](k) = \begin{bmatrix} x_1[m](k) \\ \vdots \\ x_N[m](k) \end{bmatrix} \tag{3.1}$$

Após o processo de inicialização, a função de avaliação, chamada de *FA* (Linha 4 do Algoritmo 1), calcula a aptidão dos N indivíduos $x_j[m](k)$ da população. Essa operação é aplicada para todos os indivíduos e resulta em um respectivo valor $y_j[a](k)$ para cada j-ésimo indivíduo, onde a é o número de bits que representa o valor de aptidão. Quanto melhor o valor $y_j[a](k)$ do indivíduo $x_j[m](k)$, maior a probabilidade do mesmo prosseguir

nas novas gerações. Os valores de aptidão de todos os N indivíduos são armazenados em

$$\mathbf{Y}[a](k) = \begin{bmatrix} y_1[a](k) \\ \vdots \\ y_N[a](k) \end{bmatrix}. \tag{3.2}$$

Após o cálculo da aptidão de cada j-ésimo indivíduo da k-ésima geração é realizada a operação de seleção. No AG o propósito da seleção é destacar os indivíduos $x_j[m](k)$ com seus respectivos valores de função de avaliação, $y_j[a](k)$, com o intuito de gerar populações futuras mais aptas. Existe na literatura uma grande variedade de métodos de seleção e entre eles podem ser citados: método da seleção por ranking, torneio, roleta e elitismo. O método da seleção por torneio que foi utilizado nessa implementação, é um dos mais utilizados (Noraini & Geraghty 2011) e faz uma competição entre dois ou mais indivíduos escolhidos aleatoriamente a partir da população armazenada em $\mathbf{X}[m](k)$. Essa competição consiste em comparar as aptidões, $y_j[a](k)$, dos indivíduos participantes e o indivíduo que possui o melhor valor na função de avaliação prossegue no algoritmo para passar o seus genes adiante.

A função de seleção, chamada aqui de *FS* (Linha 7 do Algoritmo 1), possui como entrada os vetores $\mathbf{Y}[a](k)$ e $\mathbf{X}[m](k)$ da k-ésima geração e como saída a variável $w_j[m](k)$ que pode assumir o valor de qualquer um dos N indivíduos armazenados em $\mathbf{X}[m](k)$. Todos valores de $w_j[m](k)$ são agrupados em

$$\mathbf{W}[m](k) = \begin{bmatrix} w_1[m](k) \\ \vdots \\ w_N[m](k) \end{bmatrix} \tag{3.3}$$

para serem utilizados na etapa de cruzamento.

O processo de cruzamento da k-ésima geração ocorre após a seleção dos indivíduos mais aptos da população (armazenados em $\mathbf{W}[m](k)$) pela função de seleção e tem como objetivo originar novos indivíduos que após a etapa de mutação irão compor a próxima geração do AG (atualização do vetor $\mathbf{X}[m](k)$). Existem várias técnicas de cruzamento apresentadas na literatura e a estratégia adotada nesta implementação foi o cruzamento com um ponto de corte. A operação de cruzamento, chamada aqui de *FC* (Linha 10 do Algoritmo 1), possui como entrada pares de elementos do vetor $\mathbf{W}[m](k)$ da k-ésima geração e como saída pares do vetor

$$\mathbf{Z}[m](k) = \begin{bmatrix} z_1[m](k) \\ \vdots \\ z_N[m](k) \end{bmatrix} \tag{3.4}$$

o qual armazena os indivíduos após o cruzamento, ou seja, os filhos da k-ésima geração.

A última etapa do AG é caracterizada pela operação de mutação que altera o valor de um grupo P de indivíduos, a fim de permitir uma maior diversidade na população evitando a estabilização da solução em mínimos locais. A taxa de mutação, TM, é o parâmetro que

controla a quantidade de indivíduos a sofrerem mutação. Normalmente, TM varia em torno de $0,1\%$ à 2% e o valor de P pode ser facilmente calculado pela expressão

$$P = \lceil N \times TM \rceil. \tag{3.5}$$

A operação de mutação, chamada aqui de FM, é apresentada na Linha 13 do Algoritmo 1 e detalhada na Equação 3.6.

$$x = (\neg z \wedge Rand) \vee (z \wedge \neg Rand). \tag{3.6}$$

Onde z é o cromossomo a ser mutado e $Rand$ é uma variável aleatória. O resultado dessa operação de Ou exclusivo entre z e $Rand$ ($Z \oplus Rand$) resulta em x.

3.4 Função de Avaliação

A função de avaliação, também chamada de função *fitness*, função objetivo ou ainda função de aptidão, tem como propósito calcular o quão apto os indivíduos $x_j[m](k)$ da população estão para prosseguir nas novas gerações. Segundo (Haupt & Haupt 2004), a função de avaliação pode ser uma função matemática, um experimento ou até um movimento em um jogo. Os AGs utilizam o conhecimento das funções de avaliação e nenhum outro conhecimento auxiliar do problema a ser otimizado.

Em vez de usar informações previamente conhecidas ao tomar decisões e fazer alterações nos cromossomos, os AGs fazem alterações aleatórias em suas soluções candidatas e usam a função de avaliação para determinar se essas mudanças produzem uma melhora na solução ou não (Vavouras et al. 2009). Alguns autores, dependendo do problema de otimização, afirmam que a aptidão é a operação mais demorada no AG (Scott et al. 1997).

3.5 Seleção

O processo de seleção de cromossomos de uma população em um AG se baseia no processo de seleção natural descrito por Charles Darwin na teoria moderna da evolução (*Evolução das Espécies* 2015).

Na natureza, os indivíduos de uma população possuem as mais variadas características. Características essas, que podem ser atribuídas a parâmetros aleatórios escolhidos pela natureza e também pela herança genética herdada dos seus respectivos pais. De forma semelhante, as possíveis soluções de um problema em um AG possuem características distintas e também carregam heranças genéticas dos seus respectivos pais.

O processo de seleção natural descrito por Charles Darwin possui como função conservar e passar para as futuras gerações as características contidas na população que favorecem o desenvolvimento dessa em um dado ambiente. Comparativamente, os cromossomos de um AG são tão bom quanto bem adaptados forem à função ou problema a ser solucionado. Em outra palavras, os cromossomos que resultam em melhores valores de aptidão quando aplicados à função de avaliação do AG são considerados os melhores da população.

Como um exemplo didático pode-se comparar uma população de girafas com diferentes características que buscam se alimentar de frutos localizados nos topos das árvores em uma floresta. Nesse caso, os cromossomos da população seriam as girafas e o processo de seleção buscará os indivíduos com maiores pescoços, uma vez que eles terão mais facilidade para se alimentar, e aqueles mais velozes e com mais facilidade para fugir de predadores.

Analogamente em um AG, as girafas, nesse caso, seriam os cromossomos $x_j[m](k)$ da população e o conjunto de características como comprimento do pescoço e velocidade dos animais determinariam os respectivos valores, $y_j[a](k)$, saídas da função de avaliação.

Conforme foi afirmado por (Floreano et al. 2008), a seleção natural se baseia na premissa de que os indivíduos geram diversas crias em uma nova população, porém nem todos os novos indivíduos se reproduzirão. Ainda segundo o mesmo autor, a seleção dos indivíduos não é feita completamente de forma aleatória já que o meio impõe restrições que regulam esse processo.

De forma análoga, o processo de seleção nos AGs têm como objetivo escolher os melhores indivíduos $x_j[m](k)$ de acordo com seus respectivos valores de função de avaliação, $y_j[a](k)$, com o intuito de gerar populações futuras com melhores soluções ao problema a ser resolvido.

O elitismo, um mecanismo de seleção, escolhe os melhores indivíduos de acordo com o valor da sua função de avaliação na geração $(k-1)$ e os passa diretamente para a próxima geração, (k), sem passar pelas etapas de cruzamento ou mutação. O intuito desse mecanismo é preservar os indivíduos mais aptos da população e acelerar a velocidade de convergência da solução.

3.6 Cruzamento

O processo de cruzamento ocorre após a seleção dos indivíduos mais aptos da população $\mathbf{X}[m](k-1)$ pelo módulo de seleção e tem como objetivo originar novos indivíduos que irão compor a próxima geração do AG, $\mathbf{X}[m](k)$. Em resumo a operação de cruzamento pode ser definida como sendo a sequência dos seguintes eventos:

1. Seleção de dois indivíduos
2. Separação em duas partes de ambos os indivíduos em uma posição específica e aleatória
3. Troca entre as partes separadas de ambos os indivíduos
4. Junção das partes separadas de diferentes indivíduos

O passo 1 menciona a seleção de um par de indivíduos já explicada pela seção de seleção do AG . Lembrando que um indivíduo $x_j[m](k)$ pode assumir diversas formas como mencionado anteriormente. Após isso, ambos os indivíduos selecionados, comumente identificados como os pais dos novos indivíduos a serem gerados, os filhos, são divididos em duas partes. Essa divisão é chamada de corte e secciona os indivíduos $x_j[m](k)$ em partes denominadas de cabeça e cauda. Dessa forma, cada indivíduo pai passa a ter, temporariamente duas partes, uma cabeça h_j e uma cauda t_j de forma que $x_j(k) = h_j(k) +$

$t_j(k)$, onde "+" representa a operação de concatenação. A posição do corte no indivíduo é escolhida de forma aleatória e ambos os pais são cortados na mesma posição. Com a divisão feita, é hora de trocar e juntar a cabeça h_{1j} do pai um com a cauda t_{2j} do pai dois e a cabeça h_{2j} do pai dois com a cauda t_{1j} do pai um. Os detalhes c) e d) da Figura 3.1 ilustram de forma clara os processos descritos.

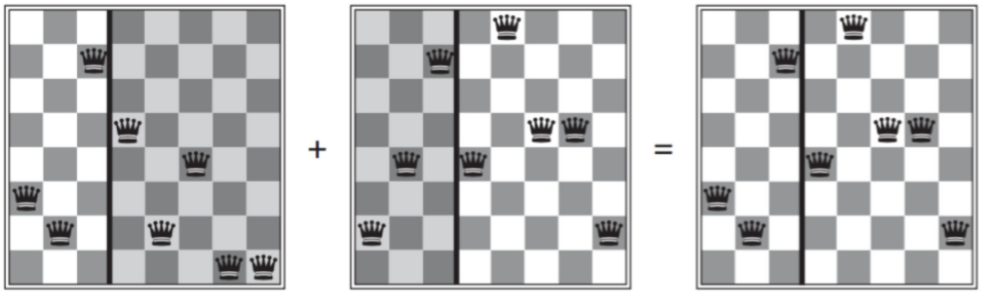


Figura 3.2: Exemplo de cruzamento de um AG onde os indivíduos são configurações das posições das peças em um jogo de xadrez. Fonte: (Russell et al. 2003).

Conforme ilustrado e detalhado por (Russell et al. 2003), a Figura 3.2 apresenta um outro exemplo de cruzamento entre dois indivíduos da população de um algoritmo genético. No caso desse algoritmo genético, o cromossomo é a configuração das posições das peças em um jogo de xadrez. Os dois primeiros tabuleiros da figura representam os dois pais responsáveis pela criação do filho que é mostrado no tabuleiro mais a direita. As zonas claras dos dois primeiros tabuleiros representam a cabeça e cauda, respectivamente utilizadas para gerar o filho apresentado. O ponto de corte utilizado nesse exemplo foi na terceira coluna, linha vertical do tabuleiro.

3.7 Mutação

Aplicada a uma recém criada geração de filhos, $W[m](k)$, originados no processo de cruzamento, a mutação altera o valor de um grupo P de indivíduos, e finaliza uma geração do algoritmo genético. Uma probabilidade fixa e pequena, é responsável por decidir a mudança aleatória e artificial do valor do indivíduo $w_j[m](k)$, mudança no valor do bit de 0 para 1, por exemplo. Em (Munakata 1998) exemplifica-se valores de taxa de mutação variando em torno de 0.1% ou, abordando de outra forma, uma mutação por mil bits transferidos.

O objetivo da mutação em um algoritmo genético é fornecer diversidade à população, dando oportunidade a novos indivíduos que poderiam não ser criados pelo processo comum de cruzamento, dependendo da população inicial gerada. A Figura 3.3 ilustra o caso de um AG que busca maximizar uma função matemática genérica, porém toda a população, representada por círculos vermelhos, está concentrada em um valor de máximo local. Esse cenário foi estabelecido após algumas gerações de seleção e cruzamento apenas. Nesse caso, a mutação serviria para diversificar a população, gerando indivíduos

fora desse mínimo local, o que poderia levar o AG a encontrar a solução ótima, caso o indivíduo mutado fosse próximo ao ponto de máximo global.

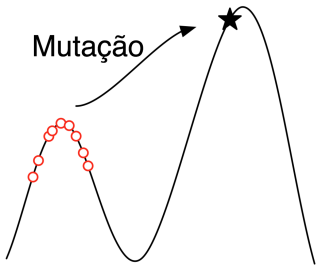


Figura 3.3: Exemplo de uma mutação em um AG. Fonte: Autoria própria.

A mutação é um artifício interessante e necessário para o sucesso do algoritmo genético, porem a frequência com que a mutação ocorre deve se manter próxima de zero, como foi explicitado anteriormente. Uma grande frequência de mutação descaracterizaria a população, dificultando a convergência do algoritmo para o ponto ótimo por meio da criação de indivíduos com baixo valor na função de avaliação e alterando, desnecessariamente, indivíduos já evoluídos e adaptados.

Capítulo 4

Descrição do Projeto

4.1 Descrição do Capítulo

Esse capítulo descreverá em detalhes a implementação paralela do AG em FPGA aqui proposta. Inicialmente, será feita uma descrição geral da arquitetura e, em seguida, de forma semelhante ao Capítulo 3, as etapas de função de avaliação, seleção, cruzamento e mutação do Algoritmo Genético serão descritos. No entanto, o presente capítulo irá descrever os módulos desenvolvidos em *hardware* responsáveis por implementar cada uma dessas etapas. Essa descrição será feita com o auxílio de esquemáticos e equações com o intuito de torná-la mais didática. Relações com as variáveis descritas previamente no Capítulo 3 e no Algoritmo 1 serão realizadas. Por fim, o módulo de sincronização será descrito na última seção desse Capítulo.

4.2 Descrição Geral do Projeto

A Figura 4.1 apresenta uma arquitetura geral da implementação paralela em *hardware* do AG proposto nesse trabalho. Todo o algoritmo foi desenvolvido utilizando uma arquitetura paralela com o intuito de acelerar a velocidade do mesmo, aproveitando os recursos disponíveis em *hardware*. A Figura detalha em diagrama de blocos os principais subsistemas da implementação proposta, que por sua vez, foram encapsulados com o intuito de tornar a visualização geral da arquitetura menos complexa. A notação utilizada nos diagramas a seguir estarão na forma $x[m](k)$, onde x é a variável, m é o número total de bits da variável e k representa a geração do algoritmo genético que vai de 1 até K. Em alguns casos pode-se usar apenas a notação entre chaves, $[m]$, para representar a quantidade de bits transferidos no barramento.

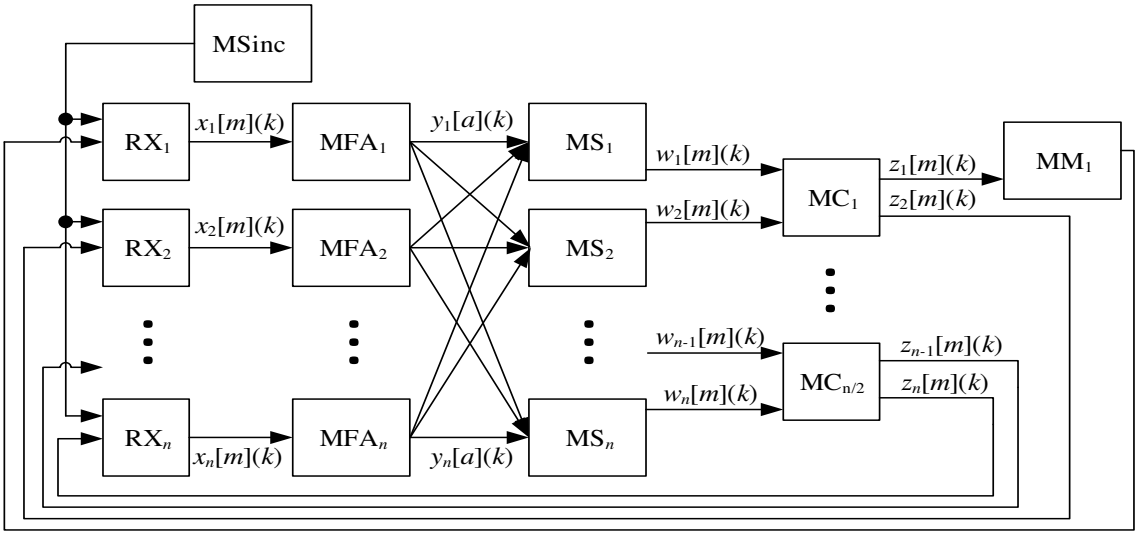


Figura 4.1: Arquitetura geral da proposta de implementação paralela de algoritmo. Fonte: Autoria própria.

Todos os valores aleatórios foram criados por geradores de números pseudo-aleatórios baseados em registradores de deslocamento do tipo *Linear Feedback Shift Register* (LFSR) (Deliparaschos et al. 2008) e (Goresky & Klapper 2006). Foram utilizados LFSR's independentes de 32 bits baseados no polinômio $r^{32}+r^{22}+r^{2}+1$ (Goresky & Klapper 2006). A cada k-ésima geração do AG uma variável aleatória de 32 bits é produzida por cada gerador de números pseudo-aleatórios. Para evitar uma mesma sequencia de valores, cada gerador de números pseudo-aleatórios possui um valor inicial de 32 bits diferente, chamado de semente.

A população inicial é também escolhida de forma aleatória, porém não pelos registrados de deslocamento, e sim, valores atribuídos pelo operador no início do AG.

4.3 Módulo de Função de Avaliação - MFA

O Módulo da Função de Avaliação tem como objetivo calcular o valor da aptidão de cada j-ésimo individuo a partir de uma função de avaliação $f(\cdot)$. A estrutura proposta possui N módulos do tipo MFA e cada j-ésimo módulo, chamado aqui de MFA_j, é associado a um individuo $x_j[m](k)$ e gera como saída em cada k-ésima geração um valor de aptidão em ponto fixo expresso por

$$y_j[a](k) = \text{MFA}j(x_j[m](k)), \tag{4.1}$$

onde a representa o número total de bits (equivalente a linha 4 do Algoritmo 1).

Não só para o Módulo de Função de Avaliação, mas para todas as outras etapas do projeto, a arquitetura proposta é capaz de solucionar problemas de uma ou duas variáveis. Independentemente do caso, o usuário não necessitará fazer nenhum ajuste no dado de

entrada. A diferença entre a quantidade de variáveis do problema a ser solucionado reflete apenas na forma como o dado é manipulado pelos módulos subsequentes, porém isso não altera o desempenho do sistema e é feito de forma invisível ao operador.

A Figura 4.2 detalha o funcionamento do j-ésimo MFA . O valor de entrada do MFA, $x_j[m](k)$ armazenado no RX_j registrador, é dividido em duas metades de $\frac{m}{2}$ bits, $px_j[\frac{m}{2}](k)$ e $qx_j[\frac{m}{2}](k)$, pelos divisores de bits $\mathrm{MFADIV1}_j$ e $\mathrm{MFADIV2}_j$ para que assim seja possível operar cada variável independentemente no caso de problemas com duas variáveis.

A Equação 4.2 apresenta uma representação de $x_j[m](k)$ contendo duas variáveis concatenadas. Os primeiros $\frac{m}{2}$ bits $(e_1e_2...e_{\frac{m}{2}})$ representam a primeira variável armazenada e na outra metade, os bits $f_1f_2...f_{\frac{m}{2}}$, representam a segunda variável armazenada em $x_j[m](k)$. No caso de problemas com somente uma variável, apenas a metade menos significativa de $x_j[m](k)$ será utilizada. Esse caso é representado pela Equação 4.3 onde a primeira metade de bits é nula.

$$x_j[m](k) = e_1e_2...e_{\frac{m}{2}}f_1f_2...f_{\frac{m}{2}} \tag{4.2}$$

$$x_j[m](k) = 0_10_2...0_{\frac{m}{2}}f_1f_2...f_{\frac{m}{2}} \tag{4.3}$$

Após a divisão, a variável $px_j[\frac{m}{2}](k)$ é direcionada para a memória ROM $\mathrm{MFAROM1}_j$ que implementa a função α em forma de LUT e a variável $qx_j[\frac{m}{2}](k)$ é direcionada para $\mathrm{MFAROM2}_j$ que implementa a função β.

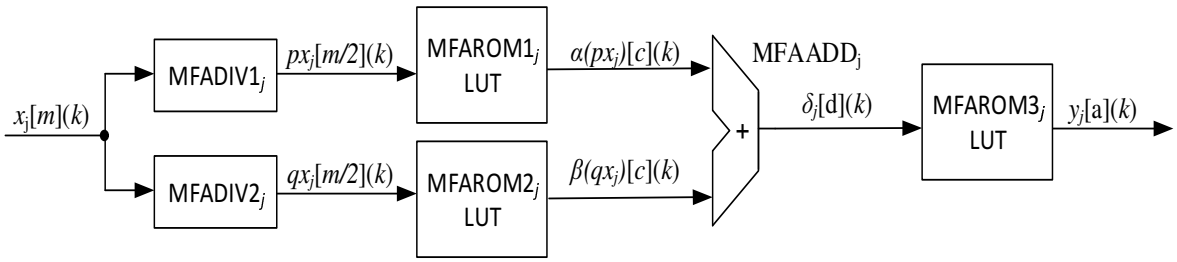


Figura 4.2: Arquitetura do j-ésimo módulo de Função de Avaliação (MFAj). Fonte: Autoria própria.

Após isso, ambos valores são somados pelo somador MFAADD_j resultando na variável $\delta_j[d](k)$, em que

$$\delta_j[d](k) = \alpha(px_j)[c](k) + \beta(qx_j)[c](k) \tag{4.4}$$

a variável $\delta_j[d](k)$ então é dirigida para a LUT $\mathrm{MFAROM3}_j$ onde será implementada a função γ, em que

$$y_j[a](k) = \gamma(\delta_j[d](k)). \tag{4.5}$$

De forma geral, o MFA apresentado na Figura 4.2 é capaz de solucionar qualquer problema de uma ou duas variáveis no formato

$$y_j[a](k) = \gamma(\alpha(px_j[c](k)) + \beta(qx_j[c](k))). \tag{4.6}$$

Expressões com produto entre as duas variáveis não são possíveis nessa abordagem atual, porém seria possível através de uma mudança na estrutura do MFA .

4.4 Módulo de Seleção - MS

O módulo de seleção (MS) implementa o método de seleção por torneio, detalhado no Capítulo 3, com competição entre dois indivíduos. Semelhantemente ao MFA existem N MS responsáveis por selecionar um grupo de N cromossomos x para popular o vetor W.

Como detalhado na Figura 4.3, cada j-ésimo MS, chamado aqui de MS_j, possui como entrada os N valores de aptidão, $y_j[a](k)$, e os N indivíduos, $x_j[m](k)$, da k-ésima geração (equivalente a linha 7 do Algoritmo 1).

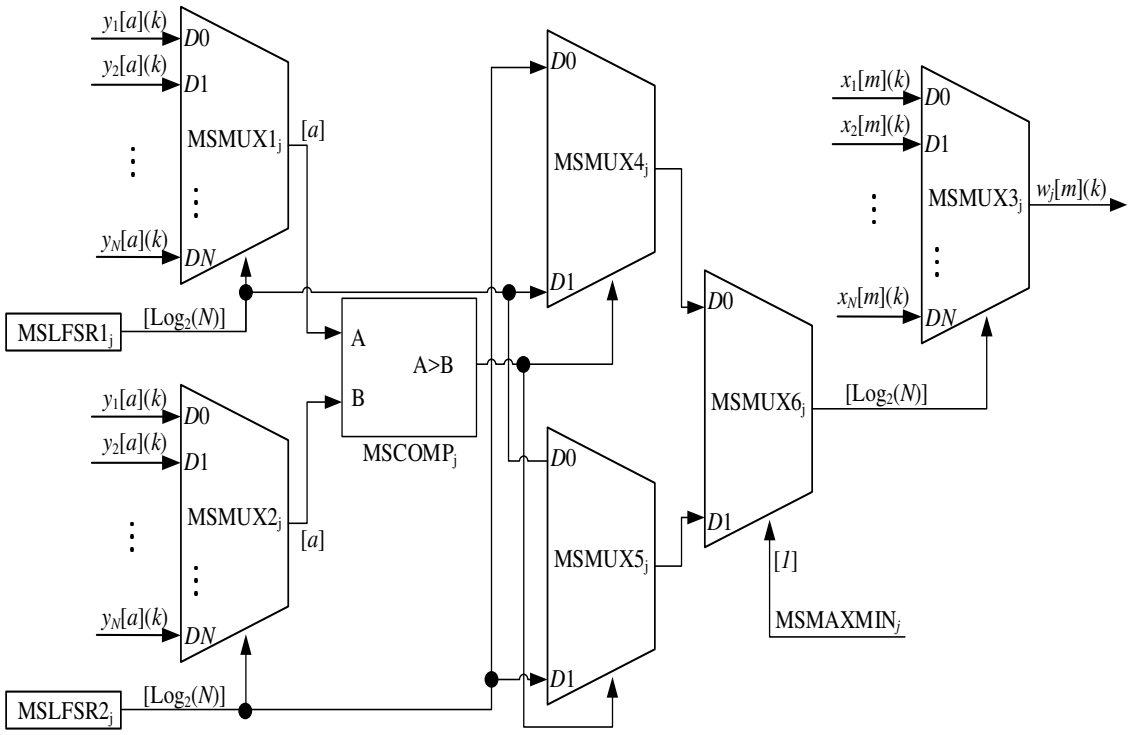


Figura 4.3: Arquitetura do j-ésimo módulo de seleção (MSj). Fonte: Autoria própria.

Cada j-ésimo MS_j possui dois geradores de números pseudo-aleatórios chamados de MSLFSR1_j e MSLFSR2_j. Além dos geradores aleatórios este módulo é formado por três multiplexadores de N entradas chamados aqui de MSMUX1_j, MSMUX2_j e MSMUX3_j, um comparador de a bits, chamado de MSCOMP_j e três multiplexadores de duas entradas, chamados de MSMUX4_j, MSMUX5_j e MSMUX6_j.

Os multiplexadores MSMUX1_j e MSMUX2_j trabalham selecionando aleatoriamente (de acordo com os valores gerados por MSLFSR1_j e MSLFSR2_j) dois índices (de 1 a N) associados aos valores de $y_j[a](k)$ (*fitness*) para participarem da seleção por torneio.

Selecionados esse dois valores, o comparador MSCOMP$_j$ irá verificar qual dos dois valores de $y_j[a](k)$ possui maior valor. Após a verificação feita pelo comparador, o índice associado ao cromossomo com o maior valor de *fitness* vai ser dirigido para a entrada $D0$ do MSMUX6$_j$, enquanto o índice associado ao cromossomo com o menor valor vai ser dirigido para a entrada $D1$. A variável booleana MSMAXMIN$_j$ irá determinar se a otimização é uma minimização ou maximização, ou seja, de acordo com a situação, essa variável vai passar adiante o índice que representa o valor maior ou menor. Finalmente, o MSMUX3$_j$ seleciona o indivíduo $x_j[m](k)$ associado ao melhor valor na função objetivo do torneio a partir da saída do MSMUX6$_j$. Saída essa que representa o índice do melhor valor de aptidão do torneio.

4.5 Módulo de Cruzamento - MC

O módulo de cruzamento detalhado aqui nessa seção implementa o cruzamento com um ponto de corte. A arquitetura aqui proposta contém $\frac{N}{2}$ módulos de cruzamento e cada um é formado por quatro divisores de bits, dois submódulos de cruzamento idênticos e dois concatenadores. De forma semelhante ao MFA descrito na Seção 4.3, o MC também possui divisores de cromossomos com intuito de manipular as duas variáveis armazenadas em $w[m](k)$ de forma independente.

Como visto na Figura 4.4, os dois cromossomos de entrada, $w_{j-1}[m](k)$ e $w_j[m](k)$, são divididos em duas metades, cada um. A primeira metade de $w_{j-1}[m](k)$ é seccionada pelo divisor MCDIV1$_j$ que passa a ser denominada como $pw_{j-1}[\frac{m}{2}](k)$ e a segunda metade dessa mesma variável é seccionada pelo divisor MCDIV2$_j$ e passa ser chamada de $qw_{j-1}[\frac{m}{2}](k)$. O mesmo acontece com o cromossomo $w_j[m](k)$ que é dividido em $pw_j[\frac{m}{2}](k)$ e $qw_j[\frac{m}{2}](k)$ pelos divisores MCDIV3$_j$ e MCDIV4$_j$, respectivamente.

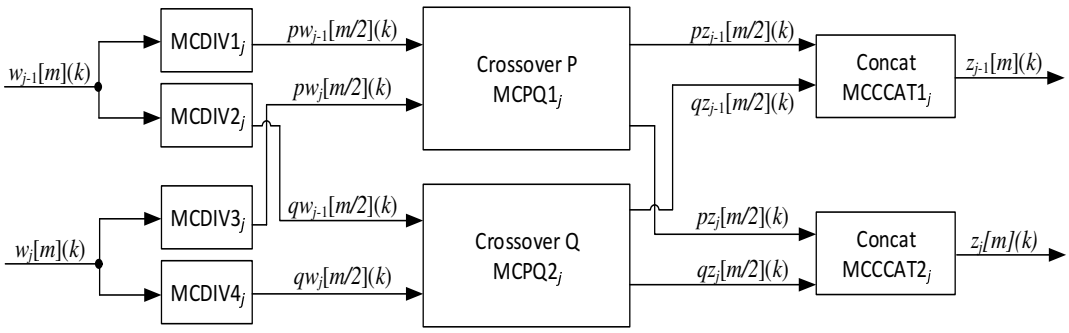


Figura 4.4: Arquitetura do j-ésimo módulo de cruzamento (MC$_j$). Fonte: Autoria própria.

Separadas as variáveis de cada cromossomo, elas são direcionadas para os submódulos MCPQ1$_j$ e MCPQ2$_j$ do MC para que só então o cruzamento seja realizado. Esse direcionamento é feito de tal forma que o cruzamento seja realizado entre variáveis semelhantes, ou seja, a primeira variável $pw_{j-1}[\frac{m}{2}](k)$ do cromossomo $w_{j-1}[m](k)$ será cruzada com a primeira variável $pw_j[\frac{m}{2}](k)$ do cromossomo $w_j[m](k)$.

Para o caso de problemas com apenas uma variável o sistema funciona de forma equivalente. Apenas a metade menos significativa das variáveis $w_{j-1}[m](k)$ e $w_j[m](k)$ conterá dados úteis e somente o bloco MCPQ2$_j$ processará dados não nulos.

A Figura 4.5 apresenta em detalhes o circuito do j-ésimo submódulo de cruzamento denominado MCPQ1$_j$. Ele é formado por um MUX de $\frac{m}{2}$ entradas, chamado aqui de MCPQMUX$_j$, cujo objetivo é selecionar de forma aleatória um dos $\frac{m}{2}$ possíveis pontos para o corte do cruzamento. A seleção de cada MCPQMUX$_j$ é controlada pelo gerador de número pseudo-aleatório MCPQLFSR1$_j$ cujo sinal de saída, MCPQ$r_{1i}[32](k)$, é truncado nos $\lceil log_2(\frac{m}{2}+1)\rceil$ bits mais significativos antes de entrar no seletor do MUX.

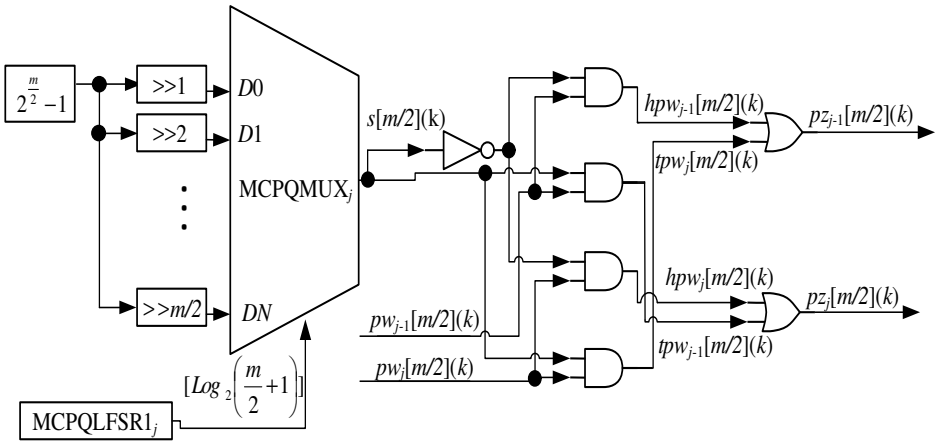


Figura 4.5: Arquitetura do j-ésimo módulo de cruzamento (MCPQ1$_j$). Fonte: Autoria própria.

A seleção do ponto de corte do MCPQ1$_j$ é feita através da máscara originada a partir da constante $2^{\frac{m}{2}}-1$. Essa constante cria um vetor de 1s do tamanho do cromossomo a ser cruzado, no caso $\frac{m}{2}$. Em seguida, um deslocamento a direita aleatório e com preenchimento de zeros é realizado de acordo com valor de MCPQLFSR1$_j$. Esse deslocamento transformará o vetor de 1s em um vetor de 0s e 1s concatenados e ainda de tamanho $\frac{m}{2}$. Essa máscara e sua inversa serão as responsáveis por realizar a operação de cruzamento auxiliada pelas portas lógicas AND e OR mostradas também na Figura 4.5.

As Equações 4.7 a 4.9 exemplificam um caso em que m = 20 e que MCPQMUX$_j$ desloca o valor de $2^{\frac{m}{2}}-1$ três vezes.

$$2^{\frac{m}{2}}-1 = 1111111111; \tag{4.7}$$

$$s_i\left[\frac{m}{2}\right](k) = 0001111111; \tag{4.8}$$

$$\neg s_i\left[\frac{m}{2}\right](k) = 1110000000. \tag{4.9}$$

Em cada k-ésima geração, as duas entradas do módulo MCPQ1$_j$, os pais $pw_{j-1}[\frac{m}{2}](k)$

e $pw_j[\frac{m}{2}](k)$ são divididas em cabeça

$$hpw_{j-1}\left[\frac{m}{2}\right](k) = \neg s_j\left[\frac{m}{2}\right](k) \wedge pw_{j-1}\left[\frac{m}{2}\right](k) \tag{4.10}$$

$$hpw_j\left[\frac{m}{2}\right](k) = \neg s_j\left[\frac{m}{2}\right](k) \wedge pw_j\left[\frac{m}{2}\right](k) \tag{4.11}$$

e cauda

$$tpw_{j-1}\left[\frac{m}{2}\right](k) = s_j\left[\frac{m}{2}\right](k) \wedge pw_{j-1}\left[\frac{m}{2}\right](k), \tag{4.12}$$

$$tpw_j\left[\frac{m}{2}\right](k) = s_j\left[\frac{m}{2}\right](k) \wedge pw_j\left[\frac{m}{2}\right](k), \tag{4.13}$$

onde $s_i[\frac{m}{2}](k)$ é a saída do MCPQMUX. Após esta etapa, o cruzamento será realizado concatenando a cabeça do pai 1, $hpw_{j-1}[\frac{m}{2}](k)$, com a cauda do pai 2, $tpw_j[\frac{m}{2}](k)$, e a cabeça do pai 2, $hpw_j[\frac{m}{2}](k)$, com a cauda do pai 1, $tpw_{j-1}[\frac{m}{2}](k)$, dando origem assim, a dois novos filhos da nova população, ou seja,

$$pz_{j-1}\left[\frac{m}{2}\right](k) = hpw_{j-1}\left[\frac{m}{2}\right](k) \vee tpw_j\left[\frac{m}{2}\right](k) \tag{4.14}$$

e

$$pz_j\left[\frac{m}{2}\right](k) = hpw_j\left[\frac{m}{2}\right](k) \vee tpw_{j-1}\left[\frac{m}{2}\right](k). \tag{4.15}$$

Para o submódulo MCPQ2$_j$ o equivalente acontece. Nesse caso, os valores de entrada serão $qw_{j-1}[\frac{m}{2}](k)$ e $qw_j[\frac{m}{2}](k)$ e as saídas serão $qz_{j-1}[\frac{m}{2}](k)$ e $qz_j[\frac{m}{2}](k)$.

Após as variáveis semelhantes terem sido cruzadas dentro de cada submódulo, elas são direcionadas às saídas dos respectivos MCPQs onde os concatenadores MCCCAT1$_j$ e MCCCAT2$_j$ irão dar origem a novos indivíduos da população concatenando ambas as partes que os formam, $pz[\frac{m}{2}](k)$ e $qz[\frac{m}{2}](k)$ (Figura 4.4).

É importante enfatizar que após os $\frac{N}{2}$ MC terem realizado as suas operações, N novos cromossomos que formarão uma nova população terão sido criados. Alguns desses indivíduos passarão pelo MM (a ser descrito na Seção 4.6) antes do início da próxima geração, porém sempre ao fim de cada iteração, N novos indivíduos terão sido criados fazendo com que a população do AG permaneça sempre com N indivíduos.

4.6 Módulo de Mutação - MM

Como no Algoritmo 1 na Linha 13 a operação de mutação será realizada em um grupo de P indivíduos, ou seja, existem P módulos de mutação e cada j-ésimo módulo, MM$_j$, altera o valor do indivíduo a ser mutado através de uma operação XOR com um número criado aleatoriamente por um gerador associado chamado de MMLFSRj (Figura 4.6). Os P MM modificarão os primeiros P indivíduos da população como mostrado na Figura 4.1.

A saída do j-ésimo, MMj, módulo em cada k-ésima geração pode ser expressa por

$$\begin{aligned} x_j[m](k+1) = &(\neg z_j[m](k) \wedge \mathrm{MM}r_j[m](k)) \\ &\vee (z_j[m](k) \wedge \neg \mathrm{MM}r_j[m](k)) \end{aligned} \tag{4.16}$$

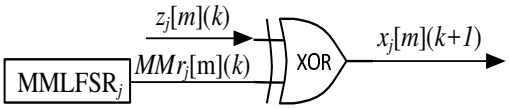


Figura 4.6: Arquitetura do j-ésimo módulo de mutação (MM$_j$). Fonte: Autoria própria.

onde MMr$_j[m](k)$ representa o número pseudo-aleatório gerado por MMLFSRj.

No caso da otimização de problemas de apenas uma variável, essa operação de mutação irá, possivelmente, atribuir valores não nulos aos $\frac{m}{2}$ bits não utilizados do cromossomo mutado. No entanto, isso não será um problema, uma vez que esses $\frac{m}{2}$ bits serão zerados ao passar pelo MFA na geração seguinte.

4.7 Módulo de Sincronização - MSinc

Por fim, o último módulo do sistema é o módulo de sincronização. Ele tem como objetivo habilitar os registradores responsáveis pelo armazenamento dos cromossomos da população do algoritmo genético a receber novos valores. Esses novos valores são resultantes dos processos de mutação e cruzamento da geração anterior e são armazenados nos registradores RX para iniciar uma nova iteração do algoritmo.

Este módulo contém um contador, um valor constante e um comparador como mostrado na Figura 4.7. A habilitação é ativada quando a comparação retorna um valor verdadeiro, ou seja, quando o valor do contador coincide com o valor armazenado na constante. O valor SyncVal [2] é obtido de acordo com o design implementado, e é ajustado de acordo com o atraso que o sistema precisa para executar todas as suas operações e disponibilizar um novo conjunto de cromossomos. O valor de saída deste módulo é um valor booleano e os valores da saída do contador e da constante são valores de 2 bits. Este número foi escolhido, pois foi o atraso máximo encontrado na implementação para uma geração inteira, um atraso para cada ROM do MFA.

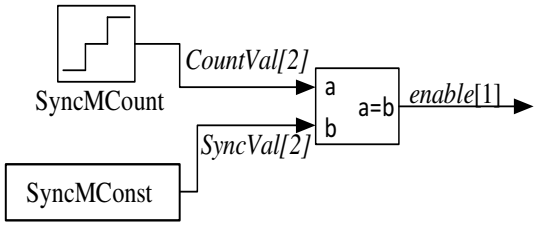


Figura 4.7: Arquitetura do Módulo de Sincronização - MSinc. Fonte: Autoria própria.

Em todos os testes realizados as operações do AG foram realizadas a uma taxa de amostragem

$$R_g = \frac{3}{T_g} \tag{4.17}$$

onde T_g é o tempo entre cada k-ésima geração.

Embora R_g seja a taxa de amostragem máxima possível de operar o sistema e T_g o tempo mínimo equivalente, a Equação 4.17 divide esse valores por 3 tendo em vista que apenas a cada três *clocks* de relógio uma nova população é originada no AG, uma vez que existem dois atrasos na arquitetura entre o início da geração k e o fim da mesma. Dessa forma, esses dois atrasos provocados pelas LUTs contidas nos MFA (Seção 4.3) fazem com que a frequência que o sistema complete o processamento de uma população $k+1$ a partir de uma população anterior k é de $\frac{3}{T_g}$.

De forma genérica, se a arquitetura contivesse quaisquer η componentes que provocassem atrasos no sistema, a taxa de amostragem $R_{g\eta}$ desse sistema seria de

$$R_{g\eta} = \frac{\eta}{T_{g\eta}}. \tag{4.18}$$

Capítulo 5

Resultados

5.1 Descrição do Capítulo

Objetivando validar a proposta de implementação do AG em FPGA foram realizadas simulações, análises e sínteses do algoritmo na otimização de diferentes funções para vários tamanhos de população. A primeira função, chamada aqui de F1, utilizada nos testes para validar a proposta foi a função de uma variável expressa como

$$f(x) = x^3 - 15x^2 + 500, \tag{5.1}$$

a segunda função, chamada aqui de F2, foi

$$f(x,y) = 8x - 4y + 1020, \tag{5.2}$$

e, por fim, a última função, chamada aqui de F3, utilizada foi a função

$$f(x,y) = \sqrt{x^2 + y^2}. \tag{5.3}$$

Dessa forma, esse capítulo apresentará os resultados experimentais obtidos, assim como análises feitas acerca dos dados produzidos pelas simulações e sínteses da arquitetura proposta aplicada às funções mencionadas acima.

5.2 Metodologia

Esse trabalho implementou e sintetizou em FPGA as três funções citadas anteriormente para populações de tamanho N = 4, N = 8, N = 16, N = 32 e N = 64 e para cromossomos com tamanho m = 20, m = 22, m = 24, m = 26, e m = 28 bits.

É importante enfatizar que essas funções foram escolhidas por motivo de comparação, uma vez que elas já foram utilizadas na literatura em trabalhos anteriores que serão mostrados nas próximas seções. No entanto, a implementação aqui proposta é capaz de implementar qualquer função no formato mostrado pela Equação 4.6 necessitando apenas da modificação dos valores armazenados nas memórias mostradas na Seção 4.3.

O desenvolvimento do projeto foi feito utilizando a plataforma de desenvolvimento fornecida pelo fabricante do FPGA Virtex 7, no caso, a Xilinx. A plataforma de desen-

volvimento chamada de *System Generator* (Xilinx 2015) permite ao usuário desenvolver sistemas utilizando a estratégia de diagrama de blocos ao invés de VHDL ou Verilog e funciona como uma extensão do *software* de manipulações matriciais, Matlab®. Essa abordagem permite o desenvolvedor manter um maior foco na arquitetura do sistema de uma maneira mais simples e sem abrir mão do controle das configurações de baixo nível.

Todos os resultados foram obtidos utilizando a plataforma de desenvolvimento *Xilinx - System Generator* citada anteriormente e um FPGA Virtex 7 xc7vx550t-1ffg1158. O FPGA Virtex 7 utilizado possui 86.600 *slices* que agrupam 692.800 registradores (*flip-flops*), 554.240 células lógicas que podem ser utilizados para implementar funções lógicas ou memórias e 2.880 células de DSP com multiplicadores e acumuladores (Xilinx 2015).

5.3 Experimentos

Como mencionado anteriormente, para a validação da implementação foram minimizadas três funções distintas. A primeira delas foi a função de uma variável apresentada na Equação 5.1 e mostrada na Figura 5.1.

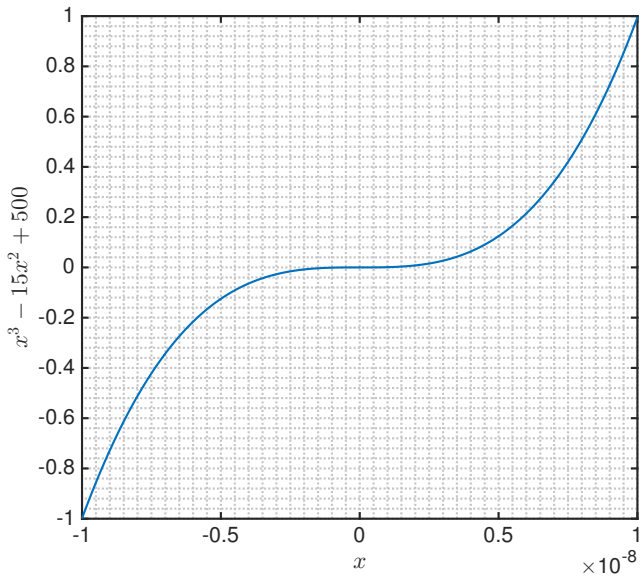


Figura 5.1: Função de Avaliação F1: $f(x) = x^3 - 15x^2 + 50$. Fonte: Autoria própria.

Essa função foi escolhida por ter sido previamente utilizada por (Vavouras et al. 2009) para validação da sua proposta de desenvolvimento de um Algoritmo Genético de alta velocidade em FPGA. Em relação a implementação dessa função de acordo como descrito na Equação 4.6, as seguintes associações podem ser feitas:

$$\alpha(px) = 0, \tag{5.4}$$

$$\beta(qx) = (qx)^3 - 15(qx)^2 + 50, \tag{5.5}$$

$$\gamma(\delta) = \delta, \tag{5.6}$$

logo, a F1 pode ser representada por

$$y(px, qx) = 1 * ((qx)^3 - 15(qx)^2 + 50 + 0). \tag{5.7}$$

A segunda função é apresentada na Figura 5.2 e foi previamente utilizada em (Fernando et al. 2008) para validar a implementação de um IP core customizável de AG para propósitos gerais.

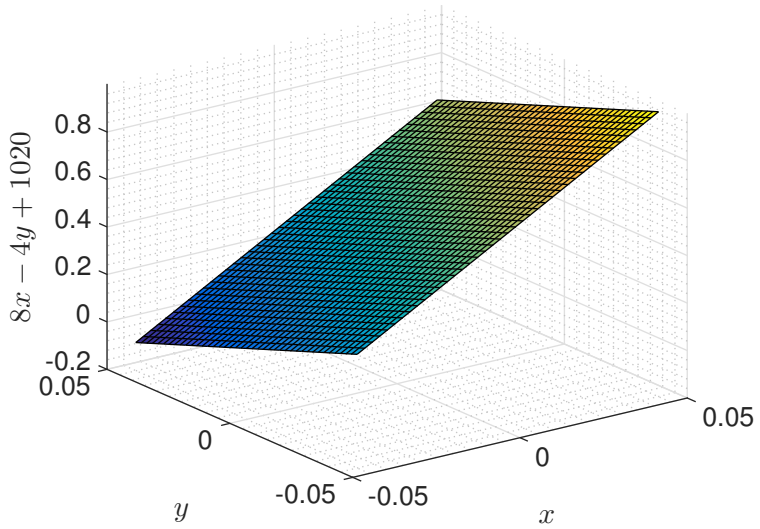


Figura 5.2: Função de Avaliação F2: $f(x,y) = 8x - 4y + 1020$. Fonte: Autoria própria.

Em relação a implementação da F2 de acordo como descrito na Equação 4.6, as seguintes associações podem ser feitas:

$$\alpha(px) = 8(px), \tag{5.8}$$

$$\beta(qx) = -4(qx) + 1020, \tag{5.9}$$

$$\gamma(\delta) = \delta, \tag{5.10}$$

logo, a F2 pode ser representada por

$$y(px, qx) = 1 * (8(px) - 4(qx) + 1020). \tag{5.11}$$

Por último a Figura 5.3 apresenta a última função utilizada para validação da proposta aqui apresentada. Essa função pôde ser vista anteriormente com o mesmo intuito em (Guo et al. 2016) e (Qu et al. 2013). Ambos os trabalhos utilizam AG, porém só (Guo et al. 2016) implementa o algoritmo em FPGA.

Semelhantemente, a F3 nos parâmetros da Equação 4.6, pode ser vista da seguinte

forma:

$$\alpha(px) = (px)^2, \tag{5.12}$$

$$\beta(qx) = (qx)^2, \tag{5.13}$$

$$\gamma(\delta) = \sqrt{\delta}, \tag{5.14}$$

logo, a F3 pode ser representada por

$$y(px, qx) = \sqrt{(px)^2 + (qx)^2}. \tag{5.15}$$

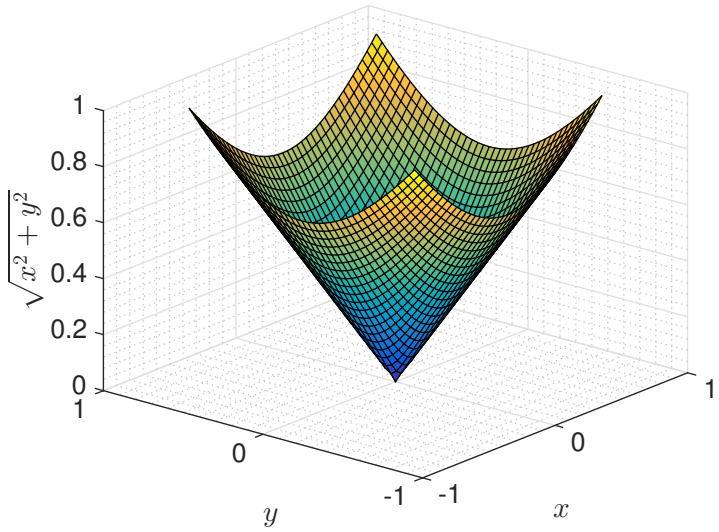


Figura 5.3: Função de Avaliação F3: $f(x,y) = \sqrt{x^2+y^2}$. Fonte: Autoria própria.

Os parâmetros para os experimentos foram baseados em configurações de experimentos anteriores encontrados na literatura em conjunto com algumas configurações obtidas empiricamente. Para o número de gerações k, notou-se, experimentalmente, que para todas as funções avaliadas, o valor mínimo procurado era obtido antes das 100 gerações do AG terem sido realizadas. Esse número está de acordo com o que é encontrado na literatura, como pode ser visto em (Vavouras et al. 2009), por exemplo. Dessa forma, $k = 100$ foi adotado como padrão para os experimentos de otimização aqui realizados.

De forma semelhante foram determinados os tamanhos de populações do AG a serem implementados e sintetizados no FPGA. Como observado em (Tommiska & Vuori 1996) o tamanho da população foi de N = 32. Já em (Scott et al. 1995) a população teve tamanho N = 16 e em (Yoshida & Yasuoka 1999) o AG foi implementado com populações de tamanho N = 64 e N = 128. Dessa forma, a arquitetura da proposta aqui apresentada foi implementada para os cinco tamanhos de população já citados na Seção 5.2. O intuito por trás desses diferentes tamanhos de N foi o de comparar o quanto esse parâmetro influencia na convergência, velocidade e ocupação em área no FPGA.

Por fim, com o mesmo intuito de comparação do quanto um parâmetro influencia em certas características de convergência e síntese, o valor da quantidade de bits m foi variado para todos os tamanhos de população como também citado na Seção 5.2. Os resultados associados serão apresentados na próxima seção.

5.4 Resultados

As Figuras 5.4 e 5.5 apresentam o funcionamento do AG aqui proposto para as funções de avaliação F1 e F3, respectivamente.

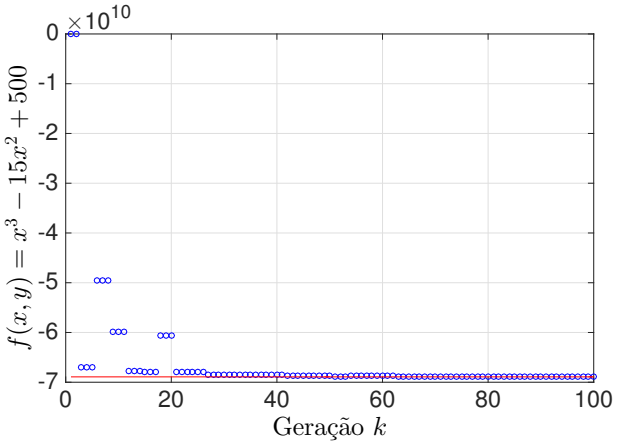


Figura 5.4: Minimização da F1. Fonte: Autoria própria

A função de avaliação 1 (Equação 5.1) foi minimizada aqui utilizando o AG com uma população de tamanho N = 32 e m =26. Dessa forma, $[\frac{m}{2}]$ bits foram utilizados para cada variável da função de avaliação. Tendo em vista que esse é um problema de apenas uma variável, a função fez uso apenas de $[\frac{m}{2}]$ bits (como na Equação 4.3). Para o teste da Figura 5.4 o intervalo de alcance da F1 foi de $f(-2^{12})$ a $f(2^{12}-1)$. Logo, o valor mínimo possível no intervalo é de $f(-2^{12}) = -6.8971 * 10^{10}$. Pela Figura, nota-se que o mínimo global foi alcançado aproximadamente na metade das 100 gerações, comprovando assim a funcionalidade do sistema proposto.

De forma semelhante, a função de avaliação 3 (Equação 5.3) também foi minimizada, como mostrado na Figura 5.5 porém com uma população de tamanho N = 64 e m =20. Nesse caso, $f(x,y)$ só admite resultados maiores ou igual a zero quando trabalha-se no domínio dos números reais, logo, o menor valor possível é o valor zero. O Algoritmo Genético paralelo implementado em FPGA aqui proposto conseguiu minimizar a F3 em um pouco mais de 20 iterações do algoritmo. Logicamente, esse não é um valor fixo, tendo em vista que o AG é um algoritmo estocástico, porém com esse valor é possível ter uma ideia da quantidade de gerações necessárias para a convergência dessa função.

Ambos os resultados mostrados foram resultantes da média de múltiplos resultados. É importante enfatizar também que parâmetros como o intervalo dos valores a serem calculados, número de bits (m), precisão em casa decimais e a possibilidade de explorar

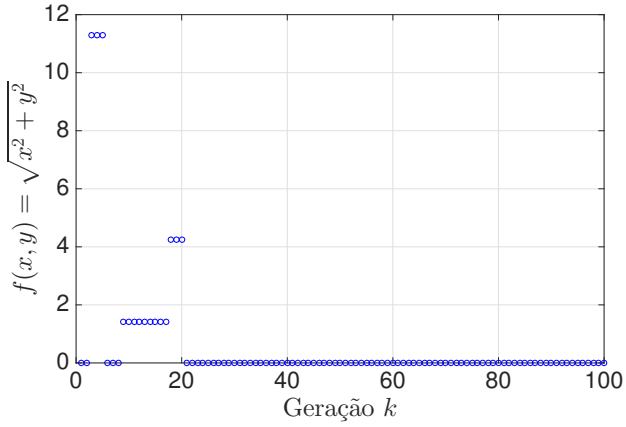


Figura 5.5: Minimização da F3. Fonte: Autoria própria.

números negativos são todos parâmetros da LUT (Seção 4.3) e configuráveis pelo usuário. Como já dito, a possibilidade de maximizar ou minimizar a função a ser otimizada também é outra variável configurável.

A Tabela 5.1 apresenta os resultados de síntese no FPGA alvo para vários tamanhos de população e $m = 20$. Observa-se claramente, em todos os casos que a área de ocupação, o *clock* e, consequentemente o *throughput*, R_g, são parâmetros muito sensíveis ao tamanho da população. Aqui, a taxa de amostragem R_g é também chamada de *throughput* pois representa a quantidade de iterações realizadas no AG por segundo, que também pode ser interpretada como possíveis soluções que o sistema fornece nesse intervalo. A Equação 4.17 afirma que o *throughput* é igual a $\frac{3}{Tg}$, ou seja, o inverso do tempo de cada geração do AG dividido por três. Isso se explica pois o sistema gera dois atrasos ao colocar duas memórias ROM em série no MFA descrito na Seção 4.3. Consequentemente, uma nova população do AG só é gerada após três *clocks* do sistema.

O *clock* é a frequência máxima desempenhada pelo sistema ao implementar essa arquitetura e ele não leva em conta os atrasos necessários para gerar uma nova população. O *clock* representa apenas a velocidade do hardware para essa específica implementação, logo, ele é $3\times$ maior que o *throughput*.

Tabela 5.1: Resultados da síntese do algoritmo genético em FPGA para $m = 20$.

N	Registradores (*flip-flops*)	Células Lógicas (LUTs)	*Clock* (MHz)	*Throughput* $- R_g$ (Msps)
4	457	592(1%)	50.28	16.76
8	839	1.558(1%)	49.32	16.44
16	1.616	4,400(1%)	49.32	16.44
32	3.225	15,908(4%)	48.51	16.17
64	6.598	58.875(16%)	34.56	11.52

A ocupação de área gasta em registradores (Figura 5.6), apresentadas na segunda co-

luna da Tabela 5.1, se deve ao armazenamento dos valores da população em RX (Figura 4.1) e dos geradores de número pseudo-aleatório, principalmente. Essa ocupação cresce linearmente de acordo com N, uma vez que quanto maior a população, maior o número de registradores RX necessários, assim como operações que necessitam dos geradores de número pseudo-aleatórios. A Figura 5.6 mostra esse crescimento de forma gráfica com uma interpolação linear.

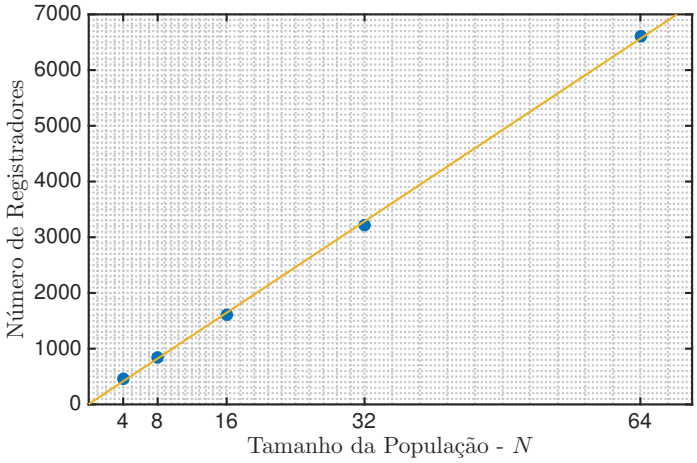


Figura 5.6: Ocupação dos Registradores no FPGA variando com N. Fonte: Autoria própria.

Já a ocupação de células lógicas (LUTs), apresentadas na terceira coluna da Tabela 5.1, foi crescente e não linear com N, como pode ser visto na Figura 5.7. Este crescimento não linear é causado pelo módulo de seleção (Subseção 4.4) que para cada j-ésimo módulo, MS_j, existem três multiplexadores de N entradas (MSMUX1_j, MSMUX2_j e MSMUX3_j).

Baseado em (Chapman 2014), cada célula lógica do Virtex 7 pode construir um MUX de 4 entradas de 1 bit, assim para montar um MUX de N entradas, são necessárias aproximadamente $\frac{N}{4}$ células lógicas totalizando valor de aproximadamente $\frac{3N}{4}$ células para cada MS_j (MSMUX4_j, MSMUX5_j e MSMUX6_j não foram contabilizados). Como existem N módulos do tipo MS tem-se aproximadamente $\frac{3N^2}{4}$ células lógicas para cada bit do barramento de entrada do MUX. Assim, explica-se o crescimento exponencial do uso das células lógicas.

Nesse contexto, é importante observar que a implementação com $N = 64$ indivíduos não atinge nem um quinto das células do FPGA (em torno de 16% do Virtex 7). Isso é um fato positivo para implementações com populações maiores.

Por último sobre a tabela, as duas últimas colunas mostram os *clocks* e os *throughputs* para cada valor de N e verifica-se uma redução da velocidade com o crescimento da população. Teoricamente, se todos os módulos fossem independentes (específicos para cada indivíduo) essa redução não deveria acontecer, porém, observa-se que nos módulos de seleção, MS_j (Figura 4.3), existe uma dependência entre os N indivíduos (devido ao

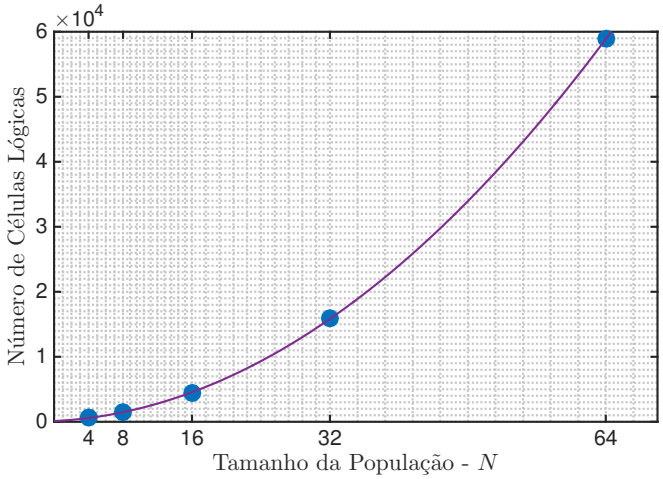


Figura 5.7: Ocupação das LUTs no FPGA variando com N. Fonte: Autoria própria.

compartilhamento de informação) causando uma junção no circuito (*join*) e com isto um aumento no tempo de processamento. Por outro lado, também observa-se que a taxa de redução não é linear, o que favorece o circuito. Outro ponto importante a ser verificado é que mesmo com a redução, cada geração do AG de 64 indivíduos é gerada em $Tg \approx 87$ ns, em outra palavras, 87 milhões de gerações a cada 1 ms. Este resultado trás um impacto bastante significativo e viabiliza o AG em várias aplicações embarcadas de tempo real como em robótica, telecomunicações e outras.

A Figura 5.8 representa a influência do número de bits m no *clock* para um AG com $N = 32$. Nota-se uma queda da velocidade de processamento com o aumento do número de bits da implementação, porém essa queda não é tão significativa. A variação do *clock* é de apenas um pouco mais de $1MHz$ quando compara-se a implementação utilizando $m = 20$ com a implementação que utiliza $m = 28$. A interpolação mostrada na figura sugere uma queda linear.

A última Figura (5.9) ilustra a relação entre o aumento de LUTs utilizadas do FPGA e a variação do número de bits m para três tamanhos diferentes de população. Nota-se uma maior diferença entre as quantidades de LUTs utilizadas em $m = 28$, principalemente, devido ao crescimento não linear desses componentes em relação a N. No entanto, como já visto na Figura 5.8 o aumento de m também é um fator responsável por diminuir a velocidade de processamento, Rg.

Analisando os resultados de síntese, notou-se que diferentes funções de avaliações como F1, F2 e F3 não resultaram em diferenças significativas de consumo de LUTs e Registradores no FPGA, bem como não foram notadas diferenças significativas em Rg. Esse resultado já era esperado, uma vez que a única variação que ocorre ao alterar a função objetivo é o conteúdo das LUTs do MFA. Sendo assim, pode-se extender esse pensamento e afirmar que os valores da Tabela 5.1 são verdadeiros para qualquer outra função, nos parâmetro da Equação 4.6, utilizando $m = 20$ bits.

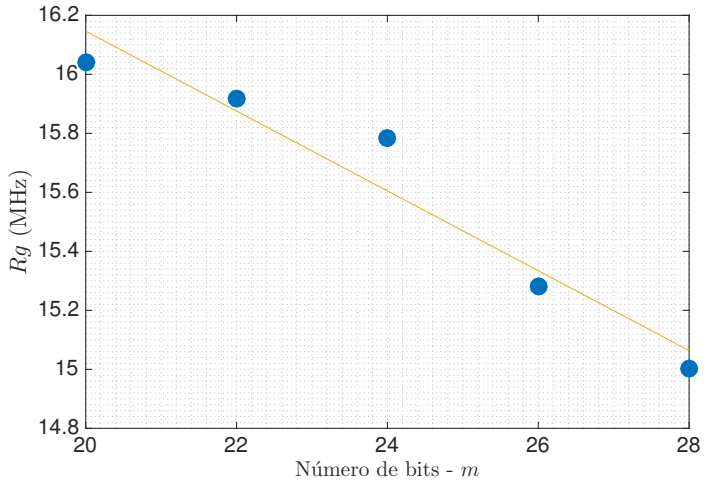


Figura 5.8: Queda de *Rg* ao variar *m*. Fonte: Autoria própria.

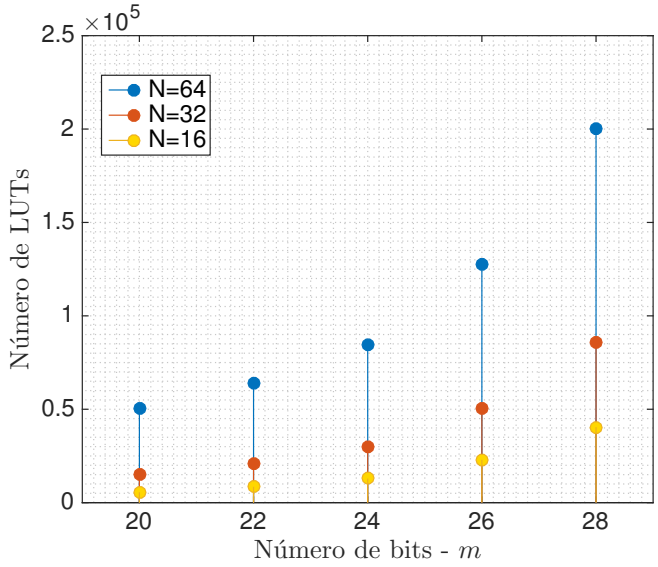


Figura 5.9: Relação entre o uso de LUTs com o aumente de *m*. Fonte: Autoria própria.

5.5 Comparações com trabalhos do estado da arte

Tomando como base os resultados apresentados na seção anterior, essa seção tem o intuito de fazer comparações dos resultados obtidos pela implementação aqui proposta com resultados equivalentes encontrados em obras pertencentes ao estado da arte. As comparações que serão mostradas a seguir e que estão resumidas na Tabela 5.2 foram feitas com a maior similaridade de parâmetros quanto possível. A tabela apresenta uma coluna que mostra as referências comparadas, as duas colunas seguintes mostram os parâmetros dos AGs comparados, em seguida os tempos obtidos pelos trabalhos do estado da arte

são mostrados e, por fim, os resultados obtidos pela implementação aqui apresentada e os respectivos *speedups* alcançados são exibidos.

O sistema apresentado por (Vavouras et al. 2009), uma implementação de alta velocidade de AG em FPGA, demonstrou um tempo de execução de $0,21$ milissegundos para um AG implementado em FPGA com $k = 100$ gerações e uma população de tamanho $N = 32$. Para as mesmas configurações, o sistema aqui proposto alcançou um tempo de $\approx 6,18$ microssegundos, o que demonstra ser $\approx 34\times$ mais rápido.

Semelhantemente, a implementação apresentada por (Yoshida & Yasuoka 1999) também apresentou um AG em FPGA com população de tamanho $N = 32$, tamanho do cromossomo de $m = 16$ e $k = 60$ gerações. A implementação validou sua proposta com o problema do caixeiro viajante e resultou em um tempo de execução de $1,702ms$. Embora não tenha sido feito nesse trabalho aqui apresentado um teste exatamente nos mesmos parâmetros de (Yoshida & Yasuoka 1999), ainda sim uma comparação pode ser feita devido à versatilidade de problemas que podem ser resolvidos por diferentes programações das LUT mostradas no MFA. Um AG com $k = 60$, $N = 32$ e $m = 20$ pode ser resolvido em $\approx 3,71$ microssegundos no trabalho aqui apresentado, significando um tempo $\approx 459\times$ mais rápido do que em (Yoshida & Yasuoka 1999).

Ainda na mesma linha, o trabalho de (Fernando et al. 2008) apresentou um IP core de AG altamente programável em FPGA. Para uma configuração de $k = 32$ e $N = 32$ os autores afirmaram um *speedup* de $5.16\times$ sobre uma implementação equivalente em *software* que alcançou um tempo de execução de $37,615$ milissegundos. Em uma comparação, a implementação aqui apresentada executa a situação equivalente em um tempo de $\approx 1,98$ microssegundos, o que apresenta um *speedup* de $\approx 19007\times$ sobre a implementação serial mostrada em (Fernando et al. 2008). Em outras palavras, uma velocidade $\approx 3683\times$ menor do que a implementação de (Fernando et al. 2008).

Por fim, a implementação aqui proposta também pode ser comparada ao trabalho publicado por (Zhu et al. 2007). Como já mencionado na Seção 1.1, esse artigo apresenta o OIMGA, uma implementação de um algoritmo monogenético em FPGA que retém apenas o melhor cromossomo da geração. Em um dos testes para validar a implementação, os autores otimizam uma função de uma variável com uma população de $N = 64$ em $\approx 0,8$ segundos. Para que o AG paralelo aqui proposto levasse esse tempo para resolver a mesma função ele iria precisar de $k = 9{,}2$ milhões de gerações. Logicamente, esse valor é absurdo. Como mostrado anteriormente nos resultados, $k = 100$ gerações foi o valor padrão para otimizar funções de uma ou duas variáveis, sendo assim, ainda que o número de gerações necessárias para otimizar a mesma função fosse de $k = 500$ (uma estimativa generosa), o tempo resultante da implementação proposta por (Zhu et al. 2007) ainda seria $\approx 18432\times$ maior.

Tabela 5.2: Tabela comparativa com trabalhos do estado da arte

Referência	Dispositivo	N	k	Tempo referência	Tempo obtido	Speedup
(Vavouras et al. 2009)	Virtex-II Pro	32	100	$0,21ms$	$6,18\mu s$	34
(Yoshida & Yasuoka 1999)	Desconhecido	32	60	$1,702ms$	$3,71\mu s$	459
(Fernando et al. 2008)	Virtex-II Pro	32	32	$7,29ms$	$1,98\mu s$	3683
(Zhu et al. 2007)	Cyclone II	64	500	$0,8s$	$43,40\mu s$	18432

Capítulo 6

Considerações Finais

6.1 Conclusões

Após a apresentação dos resultados na Seção 5 pode-se afirmar que a proposta de implementação foi de fato validada e cumpriu com o seu objetivo de ser uma implementação paralela de alto desempenho de um AG.

Os resultados de síntese confirmam que a presente proposta de implementação paralela de AG em FPGA é capaz de otimizar uma vasta gama de funções em um tempo viável para aplicações críticas que exigem curtas restrições de tempo ou uma grande quantidade de dados a ser processada em um curto intervalo.

Comparações com outras implementações encontradas na literatura feitas no fim do Capítulo 5 reforçam a alta velocidade alcançada pela implementação desenvolvida. Isso possibilita a utilização desse sistema em contexto comercial para aplicações como Internet Tátil, robótica, aplicações de tempo real e aplicações médicas. Adicionalmente, esse sistema provou ser uma ferramente de aceleração para qualquer sistema de hardware que faz uso de algoritmos genéticos.

Assim como o alto desempenho alcançado, o pequeno consumo em área da implementação aqui desenvolvida é uma característica notória. Isso possibilita que outros sistemas também sejam embarcados no FPGA, uma vez que o AG embarcado ocupa menos de $\frac{1}{5}$ das células lógicas do Virtex 7 usado como teste. Essa característica de pouco consumo de células lógicas é algo essencial para aplicações onde a área é a maior restrição como aplicações espaciais, por exemplo.

Os experimentos realizados comprovaram que os tamanhos de N testados são suficientes para resolver grande maioria dos problemas práticos assim como diz a literatura. Verificou-se que a duração em iterações (k) do AG não necessita ser um número maior do que algumas centenas. Provou-se que algumas poucas centenas de gerações ou até mesmo $k = 100$ é um número razoável de gerações para um AG. O parâmetro m mostrou ter grande importância, uma vez que ele afeta diretamente a velocidade de convergência do AG, a área ocupada no FPGA, a precisão da resposta, assim como o *throughput* obtido.

A implementação dos MFAs utilizando LUTs mostrou-se um grande avanço para o trabalho uma vez que eles foram responsáveis por aumentar o *throughput* obtido, diminuir a área ocupada no FPGA e tornar a implementação mais flexível possibilitando a mesma de implementar uma grande quantidade de funções.

6.2 Perspectivas Futuras

Perspectivas futuras apontam para adicionais estudos na implementação com o intuito de melhorar ainda mais o seu desempenho. O estudo do efeito da serialização parcial de algumas partes críticas da implementação no desempenho é algo a ser analisado. É possível que o compartilhamento de certo módulos para mais de um cromossomo resulte em algum efeito positivo no desempenho do sistema. Outra contribuição adicional válida para o trabalho são testes de diferentes técnicas de cruzamento, seleção e mutação. Existem diversas técnicas para cada um desses operadores citados anteriormente. É possível que algumas em específico sejam ainda mais apropriadas para implementações paralelas em hardware. Uma sugestão final de estudos adicionais em relação a esse projeto é a análise da diminuição do consumo energético ao diminuir o *clock* do sistema em aplicações que não necessitem da velocidade máxima obtida pela implementação.

Feito esses testes adicionais e demais melhorias, trabalhos futuros apontam para testes finais na implementação com o intuito de verificar e validar o uso da mesma para aplicações comerciais. Isso será possível através do desenvolvimento de um IP core contendo a implementação aqui apresentada.

Referências Bibliográficas

Alzubaidi, A., G. Cosma, D. Brown & A. G. Pockley (2016), Breast cancer diagnosis using a hybrid genetic algorithm for feature selection based on mutual information, *em* '2016 International Conference on Interactive Technologies and Games (ITAG)', pp. 70–76.

Amabis, José Mariano & Gilberto Rodrigues Martho (2010), 'Biologia, volume 3', *Biologia das* .

Azarian, Ali & Mahmood Ahmadi (2009), Reconfigurable computing architecture survey and introduction, *em* 'Computer Science and Information Technology, 2009. ICCSIT 2009. 2nd IEEE International Conference on', IEEE, pp. 269–274.

Chapman, K. (2014), 'Multiplexer design techniques for datapath performance with minimized routing resources', Application Note: Spartan-6 Family, Virtex-6 Family, 7 Series FPGAs.

Chen, Yajuan & Qinghai Wu (2011), Design and implementation of pid controller based on fpga and genetic algorithm, *em* 'Electronics and Optoelectronics (ICEOE), 2011 International Conference on', Vol. 4, IEEE, pp. V4–308.

Compton, Katherine & Scott Hauck (2002), 'Reconfigurable computing: a survey of systems and software', *ACM Computing Surveys (csuR)* **34**(2), 171–210.

da Silva, Lucileide Medeiros Dantas (2016), Proposta de arquitetura em hardware para fpga da técnica q-learning de aprendizagem por reforço, Dissertação de mestrado, Universidade Federal do Rio Grande do Norte.

Deliparaschos, KM, GC Doyamis & SG Tzafestas (2008), 'A parameterised genetic algorithm ip core: Fpga design, implementation and performance evaluation', *International Journal of Electronics* **95**(11), 1149–1166.

Evolução das Espécies (2015).
URL: *http://www.coladaweb.com/biologia/evolucao/evolucao-das-especies*

Fernando, Pradeep, Hariharan Sankaran, Srinivas Katkoori, Didier Keymeulen, Adrian Stoica, Ricardo Zebulum & Ramesham Rajeshuni (2008), A customizable fpga ip core implementation of a general purpose genetic algorithm engine, *em* 'Parallel and Distributed Processing, 2008. IPDPS 2008. IEEE International Symposium on', IEEE, pp. 1–8.

Floreano, Dario, Claudio Mattiussi & Bio-Inspired Artificial Intelligence (2008), 'Theories, methods, and technologies'.

Floyd, Thomas (2009), *Sistemas digitais: fundamentos e aplicações*, Bookman Editora.

Goresky, M. & A. Klapper (2006), 'Pseudonoise sequences based on algebraic feedback shift registers', *IEEE Transactions on Information Theory* **52**(4), 1649–1662.

Guo, Liucheng, Andreea Ingrid Funie, David B Thomas, Haohuan Fu & Wayne Luk (2016), 'Parallel genetic algorithms on multiple fpgas', *ACM SIGARCH Computer Architecture News* **43**(4), 86–93.

Hauck, Scott & Andre DeHon (2010), *Reconfigurable computing: the theory and practice of FPGA-based computation*, Vol. 1, Morgan Kaufmann.

Haupt, Randy L & Sue Ellen Haupt (2004), *Practical genetic algorithms*, John Wiley & Sons.

Holland, John H (1975), *Adaptation in natural and artificial systems: an introductory analysis with applications to biology, control, and artificial intelligence.*, U Michigan Press.

Instruments, National (2011), 'Understanding parallel hardware: Multiprocessors, hyperthreading, dual-core, multicore and fpgas'.
URL: *http://www.ni.com/tutorial/6097/en/*

Ionescu, Laurentiu-Mihai, Alin Mazare, Adrian-Ioan Lita & Gheorghe Serban (2015), Fully integrated artificial intelligence solution for real time route tracking, *em* '2015 38th International Spring Seminar on Electronics Technology (ISSE)', IEEE, pp. 536–540.

Jewajinda, Yutana & Prabhas Chongstitvatana (2009), Hardware architecture and fpga implementation of a parallel elitism-based compact genetic algorihm, *em* 'TENCON 2009-2009 IEEE Region 10 Conference', IEEE, pp. 1–6.

Karmali, Faisal & Mark Shelhamer (2006), Compensating for camera translation in video eye movement recordings by tracking a landmark selected automatically by a genetic algorithm, *em* 'Engineering in Medicine and Biology Society, 2006. EMBS'06. 28th Annual International Conference of the IEEE', IEEE, pp. 5298–5301.

Koza, John R (1991), 'Genetic evolution and co-evolution of computer programs', *Artificial life II* **10**, 603–629.

Li, Pei & HaiBin Duan (2012), 'Path planning of unmanned aerial vehicle based on improved gravitational search algorithm', *Science China Technological Sciences* **55**(10), 2712–2719.

Lotfi, Atieh, Abbas Rahimi, Amir Yazdanbakhsh, Hadi Esmaeilzadeh & Rajesh K Gupta (2016), Grater: An approximation workflow for exploiting data-level parallelism in fpga acceleration, *em* '2016 Design, Automation & Test in Europe Conference & Exhibition (DATE)', IEEE, pp. 1279–1284.

Mengxu, Fang & Tang Bin (2015), Fpga implementation of an adaptive genetic algorithm, *em* '2015 12th International Conference on Service Systems and Service Management (ICSSSM)', IEEE, pp. 1–5.

Merabti, Hocine & Daniel Massicotte (2014), Hardware implementation of a real-time genetic algorithm for adaptive filtering applications, *em* 'Electrical and Computer Engineering (CCECE), 2014 IEEE 27th Canadian Conference on', IEEE, pp. 1–5.

Mingas, Grigorios, Emmanouil Tsardoulias & Loukas Petrou (2012), 'An fpga implementation of the smg-slam algorithm', *Microprocessors and Microsystems* **36**(3), 190–204.

Munakata, Toshinori (1998), *Fundamentals of the new artificial intelligence*, Vol. 2, Springer.

Noraini, Mohd Razali & John Geraghty (2011), 'Genetic algorithm performance with different selection strategies in solving tsp', *World Congress on Engineering 2011 Vol II* .

Oliveira, Tiago Carvalho & Valfredo Pilla Júnior (2008), An implementation of compact genetic algorithm on fpga for extrinsic evolvable hardware, *em* 'Programmable Logic, 2008 4th Southern Conference on', IEEE, pp. 187–190.

Peng, X., X. Zheng, B. Wang, C. Zhou & Q. Zhang (2016), A micro-genetic algorithm for dna encoding sequences design, *em* '2016 2nd International Conference on Control Science and Systems Engineering (ICCSSE)', pp. 10–14.

Qu, Hong, Ke Xing & Takacs Alexander (2013), 'An improved genetic algorithm with co-evolutionary strategy for global path planning of multiple mobile robots', *Neurocomputing* **120**, 509–517.

Rodriguez, Alfonso & Felix Moreno (2015), 'Evolutionary computing and particle filtering: A hardware-based motion estimation system', *IEEE Transactions on Computers* **64**(11), 3140–3152.

Rose, Jonathan, Robert J Francis, David Lewis & Paul Chow (1990), 'Architecture of field-programmable gate arrays: The effect of logic block functionality on area efficiency', *IEEE Journal of Solid-State Circuits* **25**(5), 1217–1225.

Russell, Stuart Jonathan, Peter Norvig, John F Canny, Jitendra M Malik & Douglas D Edwards (2003), *Artificial intelligence: a modern approach*, Vol. 2, Prentice hall Upper Saddle River.

Santos, Pedro V & José C Alves (2010), Fpga based engines for genetic and memetic algorithms, *em* '2010 International Conference on Field Programmable Logic and Applications', IEEE, pp. 251–254.

Scott, S. D., A. Samal & S. Seth (1995), Hga: A hardware-based genetic algorithm, *em* 'Third International ACM Symposium on Field-Programmable Gate Arrays', pp. 53–59.

Scott, Stephen D, Sharad Seth & Ashok Samal (1997), A hardware engine for genetic algorithms, *em* 'Technical Report UNL-CSE-97-001', Citeseer.

Sehatbakhsh, Nader, Mohammad Aliasgari & Sied Mehdi Fakhraie (2013), Fpga implementation of genetic algorithm for dynamic filter-bank-based multicarrier systems, *em* 'Design & Technology of Integrated Systems in Nanoscale Era (dtis), 2013 8th International Conference on', IEEE, pp. 72–77.

Tirumalai, Vijay, Kenneth G Ricks & Keith A Woodbury (2007), 'Using parallelization and hardware concurrency to improve the performance of a genetic algorithm', *Concurrency and Computation: Practice and Experience* **19**(4), 443–462.

Todman, Timothy J, George A Constantinides, Steven JE Wilton, Oscar Mencer, Wayne Luk & Peter YK Cheung (2005), 'Reconfigurable computing: architectures and design methods', *IEE Proceedings-Computers and Digital Techniques* **152**(2), 193–207.

Tommiska, Matti & Jarkko Vuori (1996), Implementation of genetic algorithms with programmable logic devices, *em* J. T.Alander, ed., 'Proceedings of the Second Nordic Workshop on Genetic Algorithms and their Applications (2NWGA)', pp. 71–78.
URL: *ftp://ftp.uwasa.fi/cs/2NWGA/Vuori.ps.Z*

Uyemura, John P (2002), 'Introduction to vlsi circuits and systems'.

Vavouras, Michalis, Kyprianos Papadimitriou & Ioannis Papaefstathiou (2009), High-speed fpga-based implementations of a genetic algorithm, *em* 'Systems, Architectures, Modeling, and Simulation, 2009. SAMOS'09. International Symposium on', IEEE, pp. 9–16.

Xilinx (2015), 'Xilinx system generator user's guide', `www.xilinx.com`.

Yoshida, N. & T. Yasuoka (1999), Multi-gap: parallel and distributed genetic algorithms in vlsi, *em* 'Systems, Man, and Cybernetics, 1999. IEEE SMC '99 Conference Proceedings. 1999 IEEE International Conference on', Vol. 5, pp. 571–576 vol.5.

Zhu, Zhenhuan, David J Mulvaney & Vassilios A Chouliaras (2007), 'Hardware implementation of a novel genetic algorithm', *Neurocomputing* **71**(1), 95–106.